Imperial College London

Imperial College London

Department of Computing

ML Through the Looking Glass

Machine Learning with Lenses in Haskell

Author:
Alexandre Noel

Supervisor:
Prof. Nicolas Wu

submitted for the degree of

MEng Computing (Artificial Intelligence and Machine Learning)

June 11, 2026

Abstract

We present a Haskell implementation of the parametric lens framework for gradient-based machine learning proposed by Cruttwell et al. In this framework every component of a training pipeline—layer, loss function, and optimiser—is a *parametric lens*: a composable unit carrying a forward pass and a manually derived backward pass as a single algebraic structure. Composition of lenses corresponds directly to the wiring of modules in a computation graph, and the combined parameter type is inferred automatically by the type system.

The implementation extends the original Python proof-of-concept in several directions. Tensor shapes, mini-batch sizes, compute devices, and element dtypes are encoded as GHC type-level parameters, so architectural mismatches are compile-time errors rather than runtime failures; all models are written in batched form from the ground up, whereas the original processes a single example at a time. Autoencoders are expressed as two ordinary lens pipelines composed end-to-end, demonstrating that the framework is not limited to discriminative tasks. Residual skip connections are realised as a higher-order combinator `skipPara`, built entirely from existing lens primitives, with the identity gradient path emerging automatically from the combinator structure rather than from a separate derivation. Scaled dot-product self-attention and its multi-head generalisation are implemented as `ParaLens'` values with fully hand-derived backward passes, including the rank-one Jacobian correction for the softmax non-linearity, with the head-count and embedding-dimension compatibility constraint enforced at the type level. Variable-depth residual networks are constructed by typeclass induction over type-level Peano naturals. Zero overhead is verified by GHC Core inspection: the entire lens abstraction—composition, reparametrisation, the van Laarhoven `forall`—is absent from the optimised output.

Contents

Chapter 1

Introduction

1.1 The Need for a Unifying Perspective

The dominant paradigm in modern artificial intelligence, particularly deep learning, relies heavily on gradient-based optimisation techniques [1]. While these methods have achieved remarkable success across various scientific and technological domains, the ever-increasing complexity of both models and their underpinning algorithms come hand in hand with an increasing need for explainability and rigorous analysis. Current interpretations typically rely on heuristic combinations of distinct algorithmic components, often lacking a thorough, unified mathematical foundation that can scale with the sophistication of modern models [2].

1.1.1 Structural Fragmentation

A typical supervised learning scenario orchestrates an interaction between three seemingly distinct entities: the model, the loss map, and the optimiser [3]. In standard practice, these components are treated as independent modules: one might swap Softmax cross-entropy for focal loss, or replace basic gradient descent with adaptive methods like Adam or Nesterov momentum [4, 5].

However, this modularity is often implemented operationally rather than mathematically. Questions regarding the shared structural properties of these components, and whether they can be described by a single uniform language, remain largely unaddressed in standard frameworks. Furthermore, standard approaches usually restrict learning to continuous domains (smooth maps), often failing to generalise to discrete settings such as Boolean circuits [4].

1.1.2 Parametric Lenses

This project explores a unifying semantic framework proposed by Cruttwell et al. [4], which posits that gradient-based learning is fundamentally a composition of parametric lenses. In this view, a lens serves as a formal interface between internal parameters and external observables. It provides a mechanism to "zoom" into specific components of a

model, adjust their internal states, and propagate those changes back through the system.

This perspective abstracts the learning process into three core principles:

1. **Parameterisation:** All components—from neural weights to loss functions—are viewed as parameterised maps [4]. This lifts standard functions into a space where their internal configurations are first-class citizens.

2. **Bidirectional Information Flow:** The architecture must support both the forward propagation of predictions and the backward propagation of updates. This is captured by the categorical structure of lenses [6], where the "view" (forward pass) and "update" (backward pass) are intrinsically linked.

3. **Abstract Differentiation:** Parameter updates are driven by differentiation, modeled via Cartesian Reverse Differential Categories (CRDCs) [7].

By viewing the entire training loop as a morphism in a structured category, we move away from "black box" optimisation. Instead, the optimiser is reinterpreted not as an external control loop, but as a specific type of lens that reparameterises the model, creating a clean algebraic structure for gradient flows.

1.1.3 Modularity through Composition

The power of this formulation lies in its compositionality. Composing lenses corresponds to the physical "wiring" of sub-modules within a model. When we compose two parametric lenses, we are effectively defining how gradients should flow across the boundaries of different layers or loss functions.

This approach offers significant advantages over standard modularity:

- **Recursive Scaling:** Large-scale architectures can be treated as single lenses composed of smaller, verified lenses, allowing for a rigorous analysis of complex systems through their constituent parts.

- **Internalised Optimisation and Deep Compositionality:** In standard practice, an optimiser is an external control loop that sits outside the model. In this framework, an optimiser is reinterpreted as a reparameterisation lens. This allows for "Deep Compositionality": one can easily "wire" different optimisers into different sub-modules of a single architecture (e.g., using an Adam-lens for convolutional layers and an SGD-lens for dense layers). The resulting hybrid structure is itself a single lens, abstracting away internal complexity and presenting a uniform interface to the rest of the system.

1.1.4 Implementation in Haskell

The objective of this project is to instantiate the Cruttwell et al. framework in Haskell. Previous implementations have used Python [4]; Haskell offers a strong type system and mature libraries for optics [8, 9] that make the type-level guarantees central to this thesis expressible directly in the language.

1.2 Contributions

This thesis makes the following original contributions relative to the existing literature, and in particular to the Python proof-of-concept that accompanies Cruttwell et al. [4]:

1. **Static type safety for tensor computation.** Tensor shapes, mini-batch sizes, compute devices, and element dtypes are all encoded as type-level parameters via GHC's `DataKinds` extension and constraint synonyms. Shape mismatches, wrong batch sizes, and invalid device/dtype combinations are compile-time errors; the original Python implementation performs no such checking.

2. **Type-level variable network depth.** The `Stack.hs` module uses Peano naturals and typeclass induction to build networks of depth n whose parameter type is a fully concrete nested tuple at each n, preserving GHC's ability to specialise and unbox the entire parameter structure with zero overhead relative to a hand-written network of the same depth.

3. **Extended architecture support.** The framework is extended beyond the MLP scope of the original paper in two directions: autoencoders, where encoder and decoder are ordinary *ParaLens′* pipelines composed end-to-end via $(\bullet)$ with the bottleneck enforced at the type level; and residual networks, where skip connections are expressed as a single higher-order combinator *skipPara* built entirely from existing lens primitives, with the identity gradient path emerging automatically from the combinator structure.

4. **Attention as a `ParaLens'`.** Scaled dot-product self-attention and its multi-head generalisation are implemented with fully hand-derived backward passes, including the rank-one Jacobian correction for the softmax non-linearity. The constraint $e = h \cdot hd$ is enforced at the type level, so a head-count or embedding-dimension mismatch is a compile-time error rather than a silent shape failure at runtime.

5. **Zero-overhead evidence via GHC Core.** Compiling the lens-based forward pass and an equivalent hand-written forward pass with `-O2 -ddump-simpl` produces structurally identical worker functions: the entire `ParaLens` abstraction—

composition, reparametrisation, the van Laarhoven `forall`—is absent from the optimised output.

1.3 Thesis Outline

Chapter 2 covers the necessary background in Haskell's type system, optics (including the van Laarhoven lens representation), gradient-based learning, and Cartesian Reverse Differential Categories. Chapter 3 defines the `ParaLens` type, its composition operator (•) and the `repara` combinator. Chapters 4, 5, and 6 describe the library's layers, loss functions, and optimisers respectively. Chapter 7 presents the Iris and MNIST case studies, including the GHC Core zero-overhead evidence and the type-level depth experiment. Chapter 8 concludes and outlines directions for future work.

Source code. The full implementation is available at `https://github.com/LiquidPulsar/optics_haskell/tree/static`.

Chapter 2

Background

This chapter introduces the three bodies of knowledge that underpin the thesis: functional programming in Haskell, including the advanced type system features that enable static shape checking; the theory of optics that supplies the mathematical objects for representing neural network layers; and the fundamentals of gradient-based learning together with the categorical formalism used to reason about it.

2.1 Functional Programming in Haskell

Haskell is a purely functional, statically typed language with lazy evaluation. *Pure* functions have no side effects: a function $f :: a \to b$ maps every value of type a to a value of type b without reading or writing global state. Purity makes programs compositional by default—the behaviour of a composition $f \circ g$ is fully determined by the behaviours of f and g individually—which is the property the framework exploits to build large networks from small verified components. Five advanced type-system features support the framework: type classes for overloaded interfaces; the kind system and *DataKinds* for lifting tensor shapes to the type level; type families for computing output shapes at compile time; higher-rank polymorphism for composable lenses; and constraint synonyms for keeping signatures readable.

Type Classes

The primary abstraction mechanism is the *type class*: a class declares an interface and instances provide concrete implementations. Type class resolution is purely compile-time—GHC selects and inlines the appropriate instance, leaving no dispatch overhead at runtime. This property is crucial in Section 7.1, where the entire lens abstraction is shown to vanish in the compiled output.

The Kind System and DataKinds

Every type in Haskell has a *kind*, which is the "type of a type." Ordinary types such as *Int* have kind *Type*. A type constructor such as *Maybe* has kind $Type \to Type$: it takes one type argument before producing a concrete type.

The {-# LANGUAGE DataKinds #-} extension promotes value-level data constructors to the type level. The standard natural numbers, for instance, are promoted to kind Nat, and lists of naturals to kind $[Nat]$. The typed tensor library Hasktorch exploits this directly: a tensor on device dv, with element type dt, and shape $[m, n]$ is written:

$$example :: T.Tensor\ dv\ dt\ [m, n]$$

All three properties—device dv, element dtype dt, and shape $[m, n]$—are resolved at compile time. An attempt to multiply a $[3, 4]$ matrix by a $[5, 6]$ matrix produces a type error before any code runs. No runtime shape assertions are needed. Throughout this thesis, type-level lists of Nat appear wherever tensor shapes are tracked: kernel sizes $[kH, kW]$, convolution output dimensions, and batch sizes $[b, d]$ are all compile-time entities.

Type Families

A *type family* is a type-level function computed by the compiler.
The {-# LANGUAGE TypeFamilies #-} extension enables pattern-matching on types in closed equations:

$$\begin{aligned}&\textbf{type}\ family\ ToPeano\ (n :: Nat) :: PeanoNat\ \textbf{where}\\&\quad ToPeano\ 1 = One\\&\quad ToPeano\ n = Succ\ (ToPeano\ (n-1))\end{aligned}$$

The compiler reduces $ToPeano\ 3$ to $Succ\ (Succ\ One)$ at compile time, with no runtime cost. Type families appear throughout the framework for computing output shapes from layer parameters: the output height of a convolution with kernel k, stride s, and padding p applied to an input of height h is $\lfloor (h-k+2p)/s \rfloor + 1$, and this computation is performed by a type family so that the output shape is available for type-checking the next layer.

Higher-Rank Polymorphism

Standard Haskell polymorphism places the $\forall$ at the outermost level of a type: $id :: \forall\ a \circ a \to a$. The {-# LANGUAGE RankNTypes #-} extension allows the quantifier to appear inside a type:

$$\textbf{type}\ Lens\ s\ t\ a\ b = \forall\ f \circ Functor\ f \Rightarrow (a \to f\ b) \to s \to f\ t$$

Here $\forall\ f$ is scoped inside the synonym: any value of type $Lens\ s\ t\ a\ b$ must work for *any* $Functor$ the caller chooses. This is the van Laarhoven representation described in Section 2.4.2. The higher-rank quantifier is the reason lenses compose with plain $(\circ)$ —

the same function composes for any functor — and is also the mechanism by which the lens abstraction collapses to direct function calls when the functor is fixed at the call site (Section 7.1).

Constraint Synonyms

The {-# LANGUAGE ConstraintKinds #-} extension allows the kind *Constraint* (the kind of type class requirements) to be treated as a first-class kind. This enables *constraint synonyms*:

```
type SaneDT dv dt =
  ( T.KnownDType dt
  , T.StandardFloatingPointDTypeValidation dv dt
  , CanMMLens dv dt)
```

SaneDT dv dt is a single alias for a conjunction of three constraints. Functions that require all three can write *SaneDT dv dt* ⇒ in their context rather than enumerating each constraint separately. Constraint synonyms keep type signatures concise, and the type checker verifies the full set when a concrete *dv* and *dt* are supplied.

2.2 Neural Networks and Gradient-Based Learning

A *neural network* is a parametric function $f_\theta : A \to B$ indexed by a weight vector $\theta \in P$. The network is composed from differentiable building blocks—affine maps, activation functions, normalisation layers—each with their own local parameters. Training adjusts θ to minimise a scalar loss $\mathcal{L}(f_\theta(x), y)$ over a dataset of labelled pairs (x, y).

Gradient Descent

When P is a real vector space and $\mathcal{L}$ is differentiable, the canonical update rule is gradient descent:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}, \qquad \eta > 0.$$

Each step moves θ in the direction of steepest loss decrease. Practical optimisers—momentum, Adam, AdaGrad—modify this rule to improve convergence, but all share the same structure: a function from current parameters and a gradient signal to updated parameters. In this framework that function is itself a lens, described in Chapter 6.

Backpropagation

For a composed model $f_\theta = f^{(L)}_{\theta_L} \circ \cdots \circ f^{(1)}_{\theta_1}$, computing $\nabla_\theta \mathcal{L}$ by the chain rule is *backpropagation*. The forward pass evaluates $f_\theta(x)$ layer by layer, caching intermediate activations; the backward pass propagates a gradient signal from the output back through each layer in reverse, computing $\partial \mathcal{L} / \partial \theta_\ell$ at each step.

In this framework both passes are encoded simultaneously in a single lens: the *getter* is the forward pass and the *setter–getter* pair is the backward pass. The chain rule is realised by lens composition via ($\circ$) and ($\bullet$). No separate backward-pass data structure is needed.

Mini-Batching

In practice, the gradient is estimated on a *mini-batch*: a small subset of the dataset drawn at each step. If each sample has shape $[d]$, a batch of b samples has shape $[b, d]$, and all b forward and backward passes are performed simultaneously as a single tensor operation. GPU hardware is optimised for this regular parallelism, making batching essential for performance.

In this framework the batch size b is a type-level *Nat*. A model typed to accept $T.Tensor\ dv\ dt\ [b, d]$ cannot be accidentally applied to a tensor of a different batch size: the mismatch is a compile-time type error rather than a silent shape broadcast.

2.3 Cartesian Reverse Differential Categories

The theoretical foundation for backpropagation in this framework is the notion of a *Cartesian Reverse Differential Category* (CRDC) [4]. CRDCs provide an axiomatic account of reverse-mode automatic differentiation that is independent of any particular programming language or number system.

The key property used in this thesis is that every object A in a CRDC carries a *natural addition* $+_A : A \times A \to A$, arising from the abelian group structure that CRDCs impose. In the standard category of Euclidean spaces this is ordinary vector addition; in a category of boolean circuits it would be a different, discrete operation—the abstraction is independent of the specific category.

The training loop performs precisely this natural addition:

$$\theta_{\text{new}} \;=\; \theta_{\text{old}} + \partial\theta,$$

where $\partial\theta$ is whatever the backward pass emits. This is not an implementation convention but the structural update rule of the CRDC. Whether the step is gradient *descent* or *ascent*—and at what scale—is determined entirely by the sign and magnitude of $\partial\theta$, which the learning-rate cap (Section 5.1) controls by injecting a positive or negative seed into the backward pass. No modification to the update rule itself is required to switch between the two modes.

2.4 Lenses and Optics

Compound data structures are ubiquitous in modern programming, and their inherent modularity is key to building and reasoning with complex ensembles. Lenses[10] are one approach to solve the problem of accessing the components of such structures, also known as the *view-update problem*[11].

2.4.1 Origins

A simple lens from outer type S to inner type A (which we shall denote as $Lens'\ s\ a$ for reasons that will become apparent) can be thought of as comprising[10] a function $view :: s \to a$ which projects the component A, and a function $update :: s \to a \to s$ which accepts a new value of A and uses it to modify a given S.

$$\textbf{data}\ Lens'\ s\ a = Lens'\ \{view :: s \to a, update :: s \to a \to s\}$$

As a concrete example, consider the relationship between the following *Account* and *PhoneNumber* data types:

```
newtype PhoneNumber = PhoneNumber Int
newtype Account     = Account
    { phone     :: PhoneNumber
    , accountId :: Int
    }
acctToPhone :: Lens' Account PhoneNumber
acctToPhone = Lens' view update
  where
    view   :: Account → PhoneNumber
    view   = phone
    update :: Account → PhoneNumber → Account
    update acct num = acct { phone = num }
```

2.4.2 Van Laarhoven Lenses

It is tempting to extend our *Lens* type with a modification function $mod :: (a \to a) \to s \to s$[12]. This begs the question: at what point do we stop? We could usefully include $modM :: (a \to Maybe\ a) \to s \to Maybe\ s$, or even $modIO :: (a \to IO\ a) \to s \to IO\ s$. Before letting this get out of hand, we observe that both share the structure of a *Functor*, yielding $modF :: Functor\ f \Rightarrow (a \to f\ a) \to s \to f\ s$.

As a quick refresher, the category of *Functor*s[13, 14] supports the lifting of an arbitrary function into their type. In Haskell terms:

$$\textbf{class } \mathit{Functor}\ f \textbf{ where}$$
$$\quad \mathit{fmap} :: (a \to b) \to f\ a \to f\ b$$

By careful choice of specific *Functor*s we can recover *view*, *mod*, and *update*. Note that providing *const a* to *mod* recovers *update*, so we need only produce the first two. The key choices are [15, 16]:

$$\textbf{newtype } \mathit{Identity}\ a \quad = \mathit{Identity}\ \{\mathit{runIdentity} :: a\}$$
$$\textbf{newtype } \mathit{Const} \quad a\ b = \mathit{Const} \quad \{\mathit{getConst} \quad :: a\}$$
$$\textbf{instance } \mathit{Functor}\ \mathit{Identity} \textbf{ where}$$
$$\quad \mathit{fmap}\ f\ (\mathit{Identity}\ x) = \mathit{Identity}\ (f\ x)$$
$$\textbf{instance } \mathit{Functor}\ (\mathit{Const}\ m) \textbf{ where}$$
$$\quad \mathit{fmap}\ \_\ (\mathit{Const}\ v) = \mathit{Const}\ v$$

Applying *Identity* to *modF* yields $\mathit{modF} :: (a \to \mathit{Identity}\ a) \to s \to \mathit{Identity}\ s$, isomorphic to *mod*—all that is left is wrapping and unwrapping the *Identity*[1].

Applying *Const a* to *modF* yields $\mathit{modF} :: (a \to \mathit{Const}\ a\ a) \to s \to \mathit{Const}\ a\ s$, isomorphic to $(a \to a) \to s \to a$, which when provided *id* is exactly *view*.

Thus a *Lens′* only needs *modF*, and we can simplify to the type alias:

$$\textbf{type } \mathit{Lens'}\ s\ a = \forall\ f \circ \mathit{Functor}\ f \Rightarrow (a \to f\ a) \to s \to f\ s$$

This representation also lets GHC apply aggressive optimisations for higher-order functions, which will matter as we compose increasingly complex lenses.

It is often convenient to allow the types in the reverse direction to differ from those in the forward direction, yielding $\mathit{update} :: s \to b \to t$ for fresh types B and T:

$$\textbf{data } \mathit{Lens}\ s\ t\ a\ b = \mathit{Lens}\ \{\mathit{view} :: s \to a, \mathit{update} :: s \to b \to t\}$$

In the Van Laarhoven style [9]:

$$\textbf{type } \mathit{Lens}\ s\ t\ a\ b = \forall\ f \circ \mathit{Functor}\ f \Rightarrow (a \to f\ b) \to s \to f\ t$$
$$\textbf{type } \mathit{Lens'}\ s\ a \quad = \mathit{Lens}\ s\ s\ a\ a$$
$$\mathit{lens} :: (s \to a) \to (s \to b \to t) \to \mathit{Lens}\ s\ t\ a\ b$$
$$\quad \text{-- Note: fmap written infix}$$
$$\mathit{lens}\ \mathit{sa}\ \mathit{sbt}\ \mathit{afb}\ s = \mathit{sbt}\ s\ \langle\$\rangle\ \mathit{afb}\ (\mathit{sa}\ s)$$

[1] This has no runtime cost thanks to the semantics of `newtype` and `coerce`[17].

2.4.3 Composition and Modularity

The Van Laarhoven encoding gives composition for free. Given:

$x :: \mathit{Lens}\ s\ t\ a\ b$ -- forall f . Functor f => (a -> f b) -> (s -> f t)
$y :: \mathit{Lens}\ a\ b\ c\ d$ -- forall f . Functor f => (c -> f d) -> (a -> f b)

Standard function composition ($\circ$) has type $(b \to c) \to (a \to b) \to (a \to c)$, so $x \circ y$ unifies immediately:

-- x . y :: Lens s t c d
-- = forall f . Functor f => (c -> f d) -> (s -> f t)

No glue code required: composing lenses with ($\circ$) is simply function composition. This allows compound accessors to be built from primitives without any overhead.

2.4.4 Final Note

So far we have considered only lenses whose focus is a concrete field of a record, as with *PhoneNumber* inside *Account*. This is not a requirement, and in subsequent sections we will see that machine learning layers can themselves be expressed as more exotic lenses.

2.5 Related Work

Cruttwell et al., 2022

The categorical foundations of the framework are due to Cruttwell et al. [4], who showed that gradient-based learning is naturally modelled as composition in a category **Para**(C) of parametric morphisms over a CRDC C. They define parametric lenses, the composition rule, and the connection between backpropagation and the CRDC reverse derivative, and demonstrate the framework on small MLP examples.

The accompanying implementation is intentionally minimal: it is dynamically typed, operates on single examples rather than batches, and covers only dense layers and convolutions.

Importantly, the implementation in Python (whilst a natural choice for ease of development) does not fulfil the original intentions of the lens design in terms of efficient composition. Python imposes significant overhead due to the additional function calls required for composing our lenses, which cannot be optimised out.

Haskell offers far better support for lenses: thanks to its optimising compiler and powerful type reasoning capabilities we can eliminate all overhead vs hand-written code. The

present work instantiates the prior categorical structure in Haskell, contributing: statically typed tensor shapes via DataKinds; device and dtype polymorphism; batching as a type-level parameter; autoencoders and attention mechanisms; type-level variable network depth and zero-overhead evidence via GHC Core. These contributions are discussed in detail in Chapter 7.

Backprop as a Functor

Fong et al. [5] independently proposed viewing backpropagation as a functor between categories of learners. Their work establishes similar compositionality results for supervised learning but takes a different categorical approach: composition of learners corresponds directly to composition of update rules, without the explicit Para construction. Cruttwell et al. unify and generalise both lines of work within the CRDC framework.

Conventional Frameworks: PyTorch and JAX

The dominant operational frameworks treat neural networks as imperative programs annotated with automatic differentiation. Gradients are computed by tracing the forward computation graph at runtime and applying reverse-mode AD.

This differs from the categorical approach in three respects. First, the optimiser is a global object that maintains state separately from model parameters; mixing update rules across layers requires explicit parameter grouping. In this framework the optimiser state is embedded into the parameter type via *repara*, and per-layer optimisers arise automatically from the product structure of $(\bullet)$. Second, tensor shape errors typically surface at runtime when tensors are first combined; the present framework reports them as type errors at compile time. Third, there is no clear categorical account of what the composition of two PyTorch modules means mathematically; here, composition is literally morphism composition in $\mathbf{Para}(C)$.

Hasktorch

Hasktorch is the underlying tensor library used in this implementation. It provides Haskell bindings to LibTorch (the C++ backend of PyTorch) with a typed interface: *T.Tensor dv dt shape* tracks the device, dtype, and shape at the type level, supplying the primitive operations (matrix multiply, convolution, activation functions) that the framework layers on top of. Hasktorch is not itself a categorical ML framework; it provides the typed computational substrate.

Chapter 3

Core Implementation

3.1 From Lenses to Parametric Lenses

Building on the Van Laarhoven lenses of the previous chapter, we now introduce *parametric lenses* [4]: an extension that promotes the parameter of a computation to a first-class wire in the type. A standard lens *Lens s t a b* is stateless—it carries no mutable configuration of its own. For machine learning this is insufficient: a neural network layer holds weight matrices and bias vectors that are read during the forward pass and updated during the backward pass. Parametric lenses make these parameters structural, so that the update rule is part of the type rather than an afterthought.

3.2 Wire Diagrams

A lens *Lens A A′ B B′* has a direct operational reading as two functions: a *forward* computation $f : A \to B$ and a *backward* computation $f^* : A \times B' \to A'$. The backward pass requires the original input A, so the wire carrying A branches: one copy feeds f while the other curves to the right-hand side of f^*.

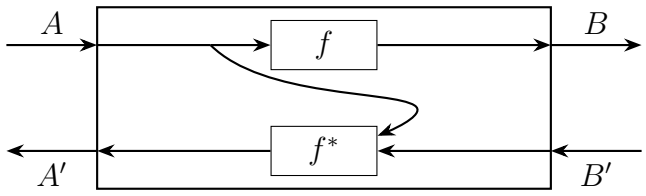

Figure 3.1: *Lens A A′ B B′* decomposed into forward pass f and backward pass f^*.

Substituting $A = (p, a)$ and $A' = (p', a')$ gives the same diagram for a *Lens* (p, a) (p', a') b b', in which the parameter p is bundled with the data on the horizontal wire:

Separating the p component onto a dedicated *vertical* axis—p entering from above on the left and p' exiting upward on the right—and retaining only a, a' on the horizontal data wires yields the canonical wire diagram for a parametric lens:

The left and right edges carry the data wires (a and b on the upper forward channels, a'

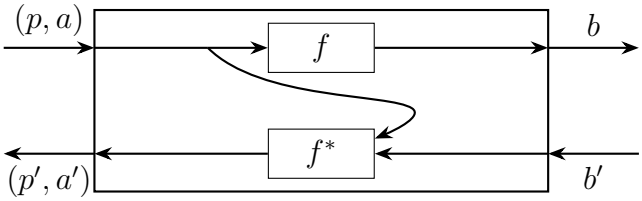

Figure 3.2: Substituting $A = (p, a)$, $A' = (p', a')$: parameter and data bundled on the horizontal wire.

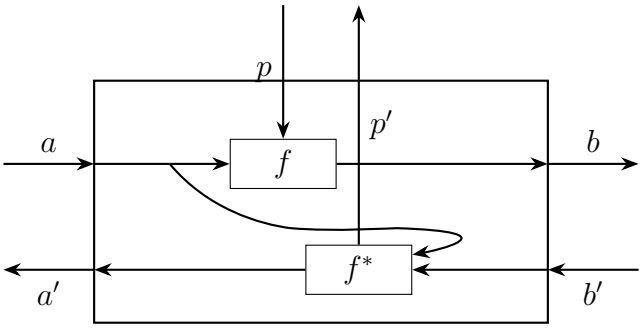

Figure 3.3: *ParaLens p p′ a a′ b b′*: p and p' factored onto the vertical axis.

and b' on the lower backward channels), and the top edge carries both parameter wires. Setting the Van Laarhoven functor to *Identity* recovers the forward pass $(p, a) \to b$; setting it to *Const* b' recovers the backward pass $(p, a, b') \to (p', a')$. The design therefore inherits lens composition, the lens library, and all compiler optimisations for free.

3.3 The *ParaLens* Type

The implementation realises the wire-diagram intuition as a type alias over the standard Van Laarhoven lens:

type *ParaLens p p′ a a′ b b′* = *Lens* (p, a) (p', a') b b'

The six type variables have the following roles:

Variable	Role
p, p'	parameter type before and after the update
a, a'	data type in the forward and backward directions
b, b'	output type in the forward and backward directions

Expanding the alias, *ParaLens p p′ a a′ b b′* is:

$$\forall f.\ \mathit{Functor}\ f \Rightarrow \Big((p, a) \to f\ b\Big) \to (p, a) \to f\ (p', a')$$

Setting $f = \mathit{Identity}$ recovers the forward pass f; setting $f = \mathit{Const}\ b'$ recovers the backward pass f^*, exactly as with ordinary lenses.

The simplified variant *ParaLens′* fixes the types to be unchanged by the pass—the natural choice for layers that do not change their own interface:

$$\textbf{type}\ \mathit{ParaLens'}\ p\ a\ b = \mathit{ParaLens}\ p\ p\ a\ a\ b\ b$$

3.4 Lifting Ordinary Lenses

A standard lens carries no parameters at all. It can be embedded into the parametric setting by equipping it with the unit parameter () via *toPara*:

$$\mathit{toPara} :: \mathit{Lens}\ a\ a'\ b\ b' \to \mathit{ParaLens}\ ()\ ()\ a\ a'\ b\ b'$$
$$\mathit{toPara} = (\mathit{leftUnit} \circ)$$

The helper *leftUnit* is the isomorphism witnessing that $((), a)$ is isomorphic to a — pairing with the unit type adds no information:

$$\mathit{leftUnit} :: \mathit{Iso}\ ((), a)\ ((), a')\ a\ a'$$
$$\mathit{leftUnit} = \mathit{iso}\ \mathit{snd}\ ((),)$$

Here $\mathit{Iso}\ s\ t\ a\ b$ is the van Laarhoven type for an isomorphism between (s, t) and (a, b), exported by `Control.Lens` [9]; $\mathit{iso} :: (s \to a) \to (b \to t) \to \mathit{Iso}\ s\ t\ a\ b$ constructs one from a pair of inverse functions. *leftUnit* is built from $\mathit{snd} :: ((), a) \to a$ (the forward direction) and $((),) :: a \to ((), a)$ (the backward direction).

Composing *leftUnit* on the left of any lens rearranges the source pair so that the trivial parameter slot disappears from the type. Symmetrically, *rightUnit* handles the case where the unit appears on the right:

$$\mathit{rightUnit} :: \mathit{Iso}\ (a, ())\ (a', ())\ a\ a'$$
$$\mathit{rightUnit} = \mathit{iso}\ \mathit{fst}\ (, ())$$

3.5 The Monoidal Product

Composition is not the only structural operation. Given a lens that acts on some component, it is frequently necessary to *extend* it to act on one component of a pair while leaving the other untouched. This is the role of *rightLens* and *leftLens*:

$$\mathit{rightLens} :: \mathit{Lens}\ p\ p'\ q\ q' \to \mathit{Lens}\ (a, p)\ (a, p')\ (a, q)\ (a, q')$$
$$\mathit{rightLens} = \mathit{alongside}\ \mathit{id}$$

$$leftLens :: Lens\ p\ p'\ q\ q' \to Lens\ (p, a)\ (p', a)\ (q, a)\ (q', a)$$
$$leftLens = flip\ alongside\ id$$

These are the two injections of the monoidal product: *alongside* from `Control.Lens` pairs two lenses into a lens on a product type. Passing *id* on one side leaves that component untouched. In wire-diagram terms, *rightLens l* adds an *extra wire on the left* that passes through the box unchanged:

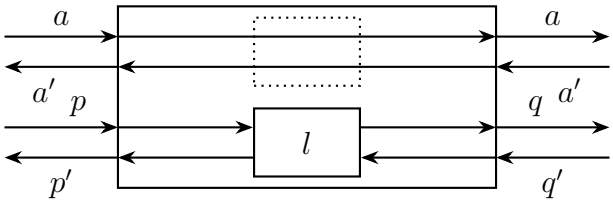

Figure 3.4: *rightLens l* (outer box): id passes a and a' through unchanged, while l transforms $p \to q$ forward and $q' \to p'$ backward.

3.6 Composition

The central operation is sequential composition, written $(\bullet)$. Given two parametric lenses whose data types are compatible:

$$f :: ParaLens\ p\ p'\ a\ a'\ b\ b'$$
$$g :: ParaLens\ q\ q'\ b\ b'\ c\ c'$$

their composition $f \bullet g$ threads the output of f into the input of g, and *pairs* the two parameter wires into a single product wire:

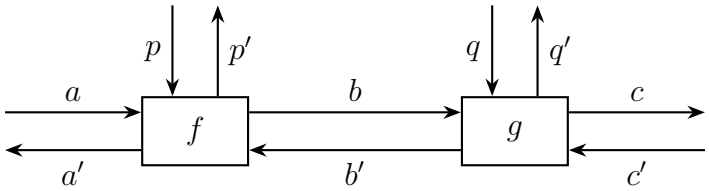

Figure 3.5: Composition $f \bullet g :: ParaLens\ (p, q)\ (p', q')\ a\ a'\ c\ c'$. The bidirectional middle channel shows b (forward, upper) and b' (backward, lower) flowing in opposite directions between the two layers.

The result has type $ParaLens\ (p, q)\ (p', q')\ a\ a'\ c\ c'$: the combined parameter is the *product* of the two individual parameter types. This directly models backpropagation: during the backward pass, gradients flow through g and then f, with each layer independently updating its own slice of the combined parameter.

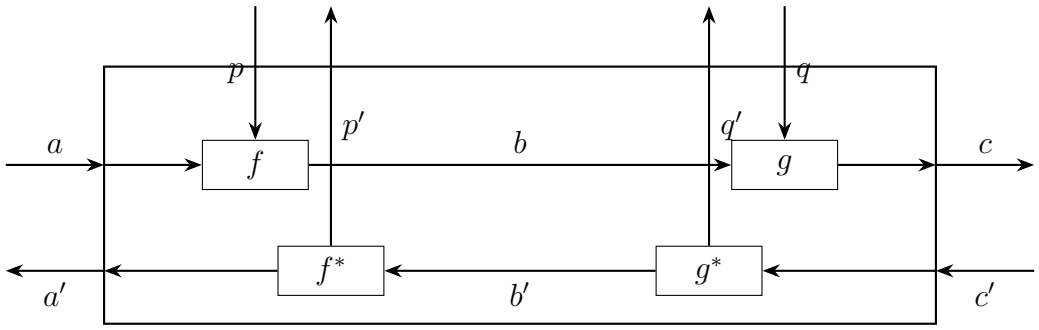

Figure 3.6: $f \bullet g$ as a single *ParaLens* (p, q) (p', q') a a' c c': each sub-lens carries its own independent parameter channel on the vertical axis, and the intermediate wire b/b' links the two halves.

The implementation uses three auxiliary isomorphisms to reassemble the nested pairs into the required shape:

infixr 8 •
(•) :: *ParaLens* p p' a a' b b'
→ *ParaLens* q q' b b' c c'
→ *ParaLens* (p, q) (p', q') a a' c c'
$f \bullet g = swapFst \circ rotate \circ rightLens\ f \circ g$

Reading right-to-left, and tracking the source type at each step:

1. $g :: Lens\ (q, b)\ (q', b')\ c\ c'$. This is the inner parametric lens mapping b to c while carrying q as a parameter.

2. *rightLens* f extends $f :: Lens\ (p, a)\ (p', a')\ b\ b'$ to act on the right slot of a pair with q on the left. Composed with g:

$$rightLens\ f \circ g\ ::\ Lens\ (q, (p, a))\ (q', (p', a'))\ c\ c'$$

3. *rotate* :: *Iso* $((a, b), c)\ \_\ (a, (b, c))\ \_$ reassociates the nested triple, changing the source from $(q, (p, a))$ to $((q, p), a)$.

4. *swapFst* :: *Iso* $((a, b), c)\ \_\ ((b, a), c)\ \_$ swaps the inner pair, changing $((q, p), a)$ to $((p, q), a)$.

The final source type is $((p, q), a)$, which matches *ParaLens* (p, q) (p', q') a a' c c' as required.

3.7 Reparametrisation

Given a lens $r :: \mathit{Lens}\ q\ q'\ p\ p'$ that relates two parameter spaces, we can *reparametrise* any $\mathit{ParaLens}\ p\ p'\ a\ a'\ b\ b'$ to run in the q parameter space:

$$
\begin{aligned}
&\mathit{repara}\ ::\ \mathit{Lens}\ q\ q'\ p\ p' \\
&\qquad \to \mathit{ParaLens}\ p\ p'\ a\ a'\ b\ b' \\
&\qquad \to \mathit{ParaLens}\ q\ q'\ a\ a'\ b\ b' \\
&\mathit{repara}\ r = (\mathit{leftLens}\ r \circ)
\end{aligned}
$$

This is the mechanism by which an *optimiser*—itself a lens that maps raw gradients to updated parameters—can be wired into a model without changing the model's own structure. Composing *leftLens r* on the left redirects the parameter wires through r before they reach the model, so the model continues to "see" its own parameter type while the outer context handles the optimiser state.

3.8 Efficiency Considerations

At first glance, the *ParaLens* type appears to introduce a lot of extra pairing and unpairing of parameters, which could be a source of overhead. However, the implementation relies on the fact that these operations are *isomorphisms* that can be optimised away by the compiler. The *swapFst*, *rotate*, and *rightLens* functions are all built from simple tuple manipulations that the Haskell compiler can inline and eliminate, resulting in no runtime overhead for the composition operation. This means that the elegant compositional structure of *ParaLens* does not come at the cost of performance, and the abstraction can be used freely without worrying about efficiency penalties.

Chapter 4

Layer Implementation

Each concrete neural network layer is a term of type *ParaLens′* or *Lens′*, directly instantiating the wire diagram of Section 3.2. The guiding principle is simple: *stateless* transformations—those with no learnable parameters—are plain *Lens′* values with no vertical parameter wires; *stateful* ones are *ParaLens′* values whose parameter type holds the learnable tensors. This distinction is enforced at the type level and is not a matter of convention.

4.1 Linear Layer and Bias

The linear layer is the framework's core building block; its backward pass establishes the gradient-flow pattern—getter computes the forward output, setter returns gradients for both weights and inputs—that every subsequent layer follows.

The weight matrix alone constitutes a *ParaLens′* whose parameter is a typed tensor of shape $[o, i]$:

$$
\begin{aligned}
&\mathit{linear} \;::\; T.\mathit{MatMulDTypeIsValid}\ \mathit{dv}\ \mathit{dt} \\
&\qquad \Rightarrow \mathit{ParaLens'} \\
&\qquad\qquad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [o, i]) \\
&\qquad\qquad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [\mathit{batch}, i]) \\
&\qquad\qquad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [\mathit{batch}, o]) \\
&\mathit{linear} = \mathit{lens}\ \mathit{fwd}\ \mathit{rev} \\
&\quad \textbf{where} \\
&\qquad \mathit{fwd}\ (w, x) = T.\mathit{matmul}\ x\ (\mathit{transp}\ w) \\
&\qquad \mathit{rev}\ (w, x)\ \mathit{grad} = (T.\mathit{matmul}\ (\mathit{transp}\ \mathit{grad})\ x,\ T.\mathit{matmul}\ \mathit{grad}\ w)
\end{aligned}
$$

The forward pass computes $xW^\top$; the backward pass applies the standard matrix-calculus identities $\partial L/\partial W = \mathit{grad}^\top x$ and $\partial L/\partial x = \mathit{grad}\, W$. The helper *transp* witnesses the transposition at the type level by permuting the first two index positions.

Bias addition is a separate *ParaLens′* that broadcasts a vector across the batch dimension and collapses it back on the reverse pass:

$$
\begin{aligned}
&\mathit{addLens} :: \mathit{CanAddLens}\ \mathit{dv}\ \mathit{dt} \\
&\qquad \Rightarrow \mathit{ParaLens'} \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ \mathit{shape}) \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ (b : \mathit{shape})) \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ (b : \mathit{shape})) \\
&\mathit{addLens} = \mathit{lens}\ \mathit{fwd}\ \mathit{rev} \\
&\quad \textbf{where} \\
&\qquad \mathit{fwd} = \mathit{uncurry}\ T.\mathit{add} \\
&\qquad \mathit{rev}\ \_ = T.\mathit{sumDim} \mathbin{@} 0 \mathbin{\&\&\&} \mathit{id}
\end{aligned}
$$

The gradient with respect to the bias is a sum over the batch axis (@0 passes the axis index to *sumDim*); the gradient with respect to the input is the identity. Composing the two with $(\bullet)$ yields a full affine layer in a single line:

$$
\begin{aligned}
&\mathit{matMulLensCore} :: \mathit{CanMMLens}\ \mathit{dv}\ \mathit{dt} \\
&\qquad \Rightarrow \mathit{ParaLens'} \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [o, i], \mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [o]) \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [\mathit{batch}, i]) \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [\mathit{batch}, o]) \\
&\mathit{matMulLensCore} = \mathit{linear} \bullet \mathit{addLens}
\end{aligned}
$$

The combined parameter $(\mathit{Tensor}\ [o, i], \mathit{Tensor}\ [o])$ arises automatically from the composition rule: the product parameter type requires no manual construction. The full backward pass—computing both $\partial L/\partial W$, $\partial L/\partial b$, and $\partial L/\partial x$—is wired together from the two constituent lenses without any additional code.

Throughout the case studies a convenience alias *matMulLens* is used, which pre-applies gradient descent to both the weight matrix and bias via *repara*:

$$
\begin{aligned}
&\mathit{matMulLens} :: (\mathit{CanMMLens}\ \mathit{dv}\ \mathit{dt}, T.\mathit{KnownDevice}\ \mathit{dv}) \\
&\qquad \Rightarrow \mathit{ParaLens'} \\
&\qquad\quad (\mathit{MMP}\ \mathit{dv}\ \mathit{dt}\ o\ i) \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [\mathit{batch}, i]) \\
&\qquad\quad (\mathit{Tensor}\ \mathit{dv}\ \mathit{dt}\ [\mathit{batch}, o]) \\
&\mathit{matMulLens} = \mathit{repara}\ (\mathit{alongside}\ \mathit{gradUpdate}\ \mathit{gradUpdate})\ \mathit{matMulLensCore}
\end{aligned}
$$

Here *MMP dv dt o i* = (*Tensor dv dt* [*o*, *i*], *Tensor dv dt* [*o*]) is the weight-bias pair type. Using *matMulLens* in a model definition is equivalent to composing *matMulLensCore* with *withGradDesc* after the fact; the difference is purely notational convenience. Chapter 6 uses *matMulLensCore* explicitly to show how any optimiser can be substituted via *repara*.

4.2 Activation Functions

Activation functions carry no learnable parameters and are plain *Lens′* values:

```
sigmoid :: T.StandardFloatingPointDTypeValidation dv dt
        ⇒ Lens′ (Tensor dv dt shape) (Tensor dv dt shape)
sigmoid = lens T.sigmoid rev
  where
    rev = (∗) ∘ ap (∗) (1−) ∘ T.sigmoid
relu    :: T.StandardFloatingPointDTypeValidation dv dt
        ⇒ Lens′ (Tensor dv dt shape) (Tensor dv dt shape)
relu = lens T.relu ((∗) ∘ heaviside)
```

The sigmoid backward pass uses $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, written point-free as $(*) \circ ap\ (*)\ (1-) \circ T.sigmoid$: the incoming gradient is multiplied element-wise by $\sigma(x)$ and $(1 - \sigma(x))$. The ReLU backward multiplies by the Heaviside step $H(x) = \mathbf{1}_{x>0}$, realised as a Boolean tensor cast to the output dtype:

```
heaviside = T.toDType @ dt @ T.Bool ∘ liftA2 (&) T.gt T.zerosLike
```

The @*dt* and @*T.Bool* type applications supply the target and source dtype to *toDType*: the Boolean comparison mask produced by *T.gt* is cast to the floating-point type *dt* that the rest of the pipeline uses.

Because these layers are plain *Lens′* values they carry no vertical parameter wires in the wire diagram sense. A *Lens′* can be appended to a *ParaLens′* pipeline using ordinary Haskell function composition (∘) rather than the parametric (•). Since *ParaLens′ p a b* = *Lens′* (*p*, *a*) *b*, the output type of any *f* :: *ParaLens′ p a b* is exactly the input type expected by any *g* :: *Lens′ b c*, so *f* ∘ *g* type-checks directly as a *ParaLens′ p a c* with the parameter type *p* unchanged:

```
affineRelu = matMulLensCore ∘ relu
```

This has the same parameter type (*Tensor* [*o*, *i*], *Tensor* [*o*]) as *matMulLensCore* alone. The alternative *matMulLensCore* • *toPara relu* would also type-check but would introduce a spurious unit into the combined parameter: (*Tensor* [*o*, *i*], *Tensor* [*o*], ()).

The operator precedence of $(\bullet)$ is set one lower than that of $(\circ)$ by design, so multi-layer stacks read cleanly without parentheses:

$$net = matMulLensCore \circ relu \bullet matMulLensCore \circ relu \bullet matMulLensCore$$

This parses as $(matMulLensCore \circ relu) \bullet (matMulLensCore \circ relu) \bullet matMulLensCore$: each $(\bullet)$ joins two already-clean $ParaLens'$ values, so the combined parameter is a nested product of weight and bias tensors only, with no unit components anywhere in the type.

4.3 Convolution

Convolution illustrates how `DataKinds`-indexed shapes extend the compile-time safety guarantee beyond simple matrix sizes: the kernel dimensions, input spatial size, stride, and padding must all be consistent, and any mismatch is a type error.

The two-dimensional convolution layer is a $ParaLens'$ with the convolutional kernel as its parameter:

$$\begin{aligned}
convLens \;::\; & (ConvSideCheck\ h\ kH\ 1\ 0\ oH \\
& , ConvSideCheck\ w\ kW\ 1\ 0\ oW \\
& , T.All\ KnownNat\ [inC, outC, h, w, batch, oH, oW] \\
& , T.KnownDType\ dt, T.KnownDevice\ dv) \\
\Rightarrow\ & ParaLens' \\
& (Tensor\ dv\ dt\ [outC, inC, kH, kW]) \\
& (Tensor\ dv\ dt\ [batch, inC, h, w]) \\
& (Tensor\ dv\ dt\ [batch, outC, oH, oW])
\end{aligned}$$

The constraints $ConvSideCheck\ h\ kH\ 1\ 0\ oH$ and $ConvSideCheck\ w\ kW\ 1\ 0\ oW$ encode the output-dimension equations

$$o_H \;=\; h - k_H + 1, \qquad o_W \;=\; w - k_W + 1$$

for unit stride and zero padding at the type level. A kernel or input of the wrong spatial size is a compile-time type error rather than a runtime failure.

The backward pass is split into two computations. The gradient with respect to the input is obtained via the transposed convolution—the adjoint of the forward operator:

$$\tfrac{\partial L}{\partial x} \;=\; convTranspose2d(kernel,\ grad)$$

The function *im2col* unfolds each spatial patch of the input into a column, producing a tensor of shape $[batch, inC * kH * kW, oH * oW]$. The kernel gradient is then a batched

matrix multiply, summed over the batch dimension:

$$\frac{\partial L}{\partial W} \;=\; \sum_{batch} grad_flat \cdot x_col^{\top} \;\in\; \mathbb{R}^{outC \times inC \cdot k_H \cdot k_W}$$

which is reshaped to $[outC, inC, kH, kW]$. This makes the gradient derivation explicit and keeps the backward pass within the verifiable part of the typed API.

4.4 Pooling and Shape

Max pooling carries no parameters and is therefore a plain *Lens′*, with kernel size, stride, and padding all verified by *ConvSideCheck* at the type level:

$$\begin{aligned} &maxPool \;::\; (ConvSideCheck\ h\ (Fst\ kernelSize)\ (Fst\ stride)\ (Fst\ padding)\ oH \\ &\qquad , ConvSideCheck\ w\ (Snd\ kernelSize)\ (Snd\ stride)\ (Snd\ padding)\ oW) \\ &\quad \Rightarrow Lens'\ (Tensor\ device\ dtype\ [batch, channels, h, w]) \\ &\qquad (Tensor\ device\ dtype\ [batch, channels, oH, oW]) \end{aligned}$$

The backward pass uses a max-routing mask: the pooled output is expanded back to the input's spatial resolution via *repeat_interleave*, compared element-wise with the original input to identify which positions achieved the maximum, and the incoming gradient flows only through those positions.

The *flatten* lens collapses all non-batch dimensions into a single axis:

$$\begin{aligned} &flatten \;::\; KnownNat\ (T.Numel\ shape) \\ &\quad \Rightarrow Lens'\ (Tensor\ dev\ dt\ (batch : shape)) \\ &\qquad (Tensor\ dev\ dt\ [batch, T.Numel\ shape]) \end{aligned}$$

The type-level computation *T.Numel shape* computes the total number of elements in the shape list at compile time, so the output width is a compile-time constant. The backward pass is a plain reshape with no gradient arithmetic. Like the activation functions, *maxPool* and *flatten* are plain *Lens′* values and compose into parametric pipelines with $(\circ)$ rather than $(\bullet)$, contributing no parameter wires to the composed diagram.

4.5 Skip Connections

A skip connection adds the input of a sub-network directly to its output, forming a residual shortcut around the learned transformation. In the parametric lens framework this is a *higher-order* combinator: given any endomorphic layer f :: `ParaLens' p a a`, the wrapped layer computes $y = f(x) + x$ and the combined parameter type is unchanged.

Gradients flow back through two paths simultaneously—one through f, one through the identity—and are summed at the input:

$$\frac{\partial L}{\partial x} = \underbrace{\frac{\partial L}{\partial y} \cdot f'(x)}_{\text{through } f} + \underbrace{\frac{\partial L}{\partial y}}_{\text{skip}}$$

The skip path ensures that gradients reach early layers even when the learned transformation f saturates or vanishes.

The implementation assembles this from two existing isomorphisms. *splitIso*::*Iso a a* (a, a) (a, a) duplicates its argument in the forward direction and sums the two results in the backward direction; *from splitIso* is the reverse: it adds in the forward direction and duplicates in the backward. (*from* :: *AnIso s t a b* → *Iso b a t s* inverts an isomorphism, swapping its two directions [9].) Together with *rightLens*, *rotate*, and *alongside*, four combinators suffice:

```
skipPara :: Num a ⇒ ParaLens' p a a → ParaLens' p a a
skipPara f = rightLens splitIso ∘ from rotate ∘ alongside f id ∘ from splitIso
```

Reading the chain left to right: *rightLens splitIso* fans the data wire a to (a, a) while leaving the parameter wire p intact; *from rotate* rearranges $(p, (a, a))$ into $((p, a), a)$ so that *alongside* can feed one copy to f—together with its parameters—and the other to *id*; the focus of *alongside f id* is the pair $(f(x),\ x)$; *from splitIso* sums the pair to produce $f(x) + x$ in the forward direction.

The backward pass is the exact mirror of the forward pass, and no additional gradient logic is needed. *from splitIso*'s *setter* distributes $\partial L/\partial y$ to both branches—precisely because its *getter* summed them in the forward direction. Symmetrically, *rightLens splitIso*'s *setter* sums the two arriving input gradients into $\partial L/\partial x$—because its *getter* fanned x to two copies going forward.

This duality is not coincidental: every Van Laarhoven lens encodes its forward computation in the getter and its inverse (in the CRDC sense) in the setter. Placing *splitIso* at one end of the chain and *from splitIso* at the other is precisely what flips their roles under the two specialisations of the functor f—getter for the forward pass, setter for the backward pass. The residual gradient formula $\partial L/\partial x = \partial L/\partial x|_f + \partial L/\partial y$ emerges for free from the lens structure; it is not written anywhere in the implementation.

A two-layer residual block is a single application of *skipPara*:

```
resBlock = skipPara (matMulLens ∘ relu • matMulLens ∘ relu)
```

The parameter type *ResBlockP dev dt* = (*MMP dev dt Hidden Hidden*, *MMP dev dt Hidden Hidden*) holds the two weight-bias pairs, inherited without modification from the inner pipeline.

Projection Skips

skipPara requires its sub-network to be *type-preserving*: $f :: ParaLens'\ p\ a\ a$. Real ResNet stages that change spatial resolution or channel count cannot satisfy this; the original paper handles such transitions with a *projection shortcut* $y = f(x) + P(x)$, where P matches the output dimension. The natural generalisation runs f and a learned projection *proj* in parallel over a shared input and sums:

$$
\begin{aligned}
&projSkipPara\ ::\ (Num\ b, Num\ b') \\
&\qquad \Rightarrow ParaLens\ p\ p'\ a\ a'\ b\ b' \\
&\qquad \to ParaLens\ q\ q'\ a\ a'\ b\ b' \\
&\qquad \to ParaLens\ (p,q)\ (p',q')\ a\ a'\ b\ b' \\
&projSkipPara\ f\ proj \\
&\quad = rightLens\ splitIso \circ r \circ alongside\ f\ proj \circ from\ splitIso
\end{aligned}
$$

The structure mirrors *skipPara*: *from splitIso* fans the input a to two copies; *alongside f proj* runs both branches in parallel, each with its own independent parameter wire; *rightLens splitIso* sums the two outputs. The auxiliary r is a bookkeeping isomorphism that rearranges the nested parameter–data pairs so that *alongside* receives the correct shape; it carries no gradient logic and is compiled away entirely.

The same *splitIso*/*from splitIso* duality applies: the gradient formula $\partial L/\partial x = \partial L/\partial x|_f + \partial L/\partial x|_P$ falls out automatically, for the same reason as in *skipPara*. Passing *toPara id* for *proj* recovers the identity-shortcut case—the skip path contributes no learnable parameters and the combined type simplifies accordingly.

4.6 Attention

Self-attention is the most structurally complex layer in this chapter: the output at every position is a weighted average of all other positions, with weights that are themselves a differentiable function of the input. Despite this non-linearity the layer is *stateful*—four square projection matrices are its learnable parameters—and it fits into the *ParaLens′* abstraction without modification. The parameter type groups the four matrices in a tuple:

```
type SelfAttnP dev dt e =
  ( Tensor dev dt [e, e]   -- Wq
  , Tensor dev dt [e, e]   -- Wk
```

```
  , Tensor dev dt [e, e]   -- Wv
  , Tensor dev dt [e, e]   -- Wo
  )
```

With batch size b, sequence length s, and embedding dimension e fixed at the type level, the signature of *selfAttention* is:

```
selfAttention :: (T.All KnownNat [b, s, e]
                  , T.MatMulDTypeIsValid dev dt
                  , T.BasicArithmeticDTypeIsValid dev dt
                  , T.StandardFloatingPointDTypeValidation dev dt
                  , T.SumDType dt~dt, T.SumDTypeIsValid dev dt
                  , KnownNat (b * s)   -- Implied
                  , (b * (s * e))~((b * s) * e))   -- Implied
                 => ParaLens'
                      (SelfAttnP dev dt e)
                      (Tensor dev dt [b, s, e])
                      (Tensor dev dt [b, s, e])
```

Forward pass. Given input $X \in \mathbb{R}^{b \times s \times e}$, scaled dot-product attention proceeds in six steps. The input is first projected into query, key, and value spaces:

$$Q = XW_Q^\top, \qquad K = XW_K^\top, \qquad V = XW_V^\top \qquad \in \mathbb{R}^{b \times s \times e}$$

The attention scores are formed by a scaled inner product, normalised by softmax, and used to produce a weighted average of the values:

$$S = \tfrac{1}{\sqrt{e}}\, QK^\top \in \mathbb{R}^{b \times s \times s}, \qquad A = \mathrm{softmax}(S), \qquad \mathit{out} = AVW_O^\top$$

All six intermediate values are retained by an internal helper *runFwd*, which is shared between the getter and the setter so that the backward pass need not recompute the forward pass:

```
runFwd (wq, wk, wv, wo) x =
  let q      = proj wq x
      k      = proj wk x
      v      = proj wv x
      scores = T.mulScalar scale $ T.matmul q $ tr k
```

$$
\begin{aligned}
&\quad \mathit{weights} = T.\mathit{softmax}\ @\ 2\ \mathit{scores} \\
&\quad \mathit{attn} \quad = T.\mathit{matmul}\ \mathit{weights}\ v \\
&\textbf{in}\ (q, k, v, \mathit{weights}, \mathit{attn}, \mathit{proj}\ \mathit{wo}\ \mathit{attn})
\end{aligned}
$$

where $\mathit{proj}\ w\ x = T.\mathit{matmul}\ x\ (T.\mathit{transpose}@0@1\ w)$ computes $xW^\top$, $tr = T.\mathit{transpose}@1@2$ transposes the last two index positions, and *scale* is the compile-time constant $1/\sqrt{e}$.

Backward pass. The setter applies the chain rule in the reverse order of the graph. At each node the standard matrix-calculus identity for $Y = XA^\top$, namely $\partial L/\partial X = (\partial L/\partial Y)\,A$ and $\partial L/\partial A = (\partial L/\partial Y)^\top X$, is applied.

Output projection. Differentiating $\mathit{out} = \mathit{attn}\,W_O^\top$:

$$
\frac{\partial L}{\partial \mathit{attn}} = \frac{\partial L}{\partial \mathit{out}}\,W_O, \qquad \frac{\partial L}{\partial W_O} = \left(\frac{\partial L}{\partial \mathit{out}}\right)^\top \mathit{attn}
$$

Both tensors are reshaped to $[b{\cdot}s,\ e]$ before the matrix multiply, yielding $\partial L/\partial W_O \in \mathbb{R}^{e\times e}$. The helper *wGrad* encapsulates this flatten-matmul pattern and is reused identically for all four projection matrices, playing the same role that $T.\mathit{matmul}\ (\mathit{transp}\ \mathit{grad})\ x$ plays in *linear*:

$$
\begin{aligned}
&\mathit{wGrad}\ x\ dY = \\
&\quad T.\mathit{matmul}\ (T.\mathit{transpose}\ @\ 0\ @\ 1\ (T.\mathit{reshape}\ @\ [\,b * s, e\,]\ dY)) \\
&\qquad\qquad (T.\mathit{reshape}\ @\ [\,b * s, e\,]\ x)
\end{aligned}
$$

Weighted sum. Differentiating $\mathit{attn} = A\,V$:

$$
\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \mathit{attn}}\,V^\top, \qquad \frac{\partial L}{\partial V} = A^\top\,\frac{\partial L}{\partial \mathit{attn}}
$$

Softmax. The Jacobian of softmax contracts to a rank-one correction. For a single row $y = \mathrm{softmax}(x)$ and incoming gradient g:

$$
\frac{\partial L}{\partial x_i} = y_i\left(g_i - \sum_j y_j\,g_j\right)
$$

The inner dot product is computed with $\mathit{sumDim}\ @\ 2$ along the key axis, reshaped to $[b, s, 1]$ for broadcasting, and the result is scaled element-wise by A:

$$
\begin{aligned}
&\mathit{softmaxBwd}\ w\ dw = w * T.\mathit{sub}\ dw\ \mathit{dot} \\
&\quad \textbf{where}\ \mathit{dot} = T.\mathit{reshape}\ @\ [\,b, s, 1\,]\ \$\ T.\mathit{sumDim}\ @\ 2\ (w * dw)
\end{aligned}
$$

Scaled dot-product. Differentiating $S = \frac{1}{\sqrt{e}} QK^\top$:

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{e}} \frac{\partial L}{\partial S} K, \qquad \frac{\partial L}{\partial K} = \frac{1}{\sqrt{e}} \Big(\frac{\partial L}{\partial S}\Big)^\top Q$$

Input projections. *wGrad* is applied once per projection to obtain $\partial L/\partial W_Q$, $\partial L/\partial W_K$, and $\partial L/\partial W_V$. The gradient with respect to the input X accumulates the three back-projections:

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Q} W_Q + \frac{\partial L}{\partial K} W_K + \frac{\partial L}{\partial V} W_V$$

The complete setter, with $mm = T.matmul$, $tr = T.transpose \mathbin{@} 1 \mathbin{@} 2$, and $scale'$ denoting multiplication by $1/\sqrt{e}$, is:

$$\begin{aligned}
&rev\ (p \mathbin{@} (wq, wk, wv, wo), x)\ dOut = ((dWq, dWk, dWv, dWo), dX) \\
&\quad \textbf{where} \\
&\qquad (q, k, v, weights, attn, _) = runFwd\ p\ x \\
&\qquad dAttn\ \ = T.matmul\ dOut\ wo \\
&\qquad dWo\ \ \ = wGrad\ attn\ dOut \\
&\qquad dWeights = mm\ dAttn\ (tr\ v) \\
&\qquad dV\ \ \ \ = mm\ (tr\ weights)\ dAttn \\
&\qquad dScores = softmaxBwd\ weights\ dWeights \\
&\qquad dQ\ \ \ \ = scale' \mathbin{\$}\ mm\ dScores\ k \\
&\qquad dK\ \ \ \ = scale' \mathbin{\$}\ mm\ (tr\ dScores)\ q \\
&\qquad dWq\ \ \ = wGrad\ x\ dQ \\
&\qquad dWk\ \ \ = wGrad\ x\ dK \\
&\qquad dWv\ \ \ = wGrad\ x\ dV \\
&\qquad dX\ \ \ \ = T.matmul\ dQ\ wq + T.matmul\ dK\ wk + T.matmul\ dV\ wv
\end{aligned}$$

Multi-head attention. *multiHeadSelfAttention* partitions the embedding dimension across h independent attention heads, each of width $hd = e/h$, enforced at the type level by $e{\sim}h * hd$. The parameter type is unchanged—the weight matrices remain $[e, e]$ and operate on the full embedding—but two reshape helpers split and merge the head dimension around the attention computation:

$$\begin{aligned}
splitHeads\ \ &= T.transpose \mathbin{@} 1 \mathbin{@} 2 \circ T.reshape \mathbin{@} [b, s, h, hd] \\
mergeHeads &= T.reshape \mathbin{@} [b, s, e] \circ T.transpose \mathbin{@} 1 \mathbin{@} 2
\end{aligned}$$

splitHeads maps $[b, s, e] \to [b, s, h, hd] \to [b, h, s, hd]$: the *reshape* partitions each embedding vector into h contiguous slices of width hd, and the *transpose* brings the head axis

adjacent to the batch axis so that subsequent batched matrix multiplies treat $b \times h$ as a single leading batch dimension. The constraint $\mathit{Numel}\ [b, s, e] \sim \mathit{Numel}\ [b, s, h, hd]$ is the type-level proof that the reshape preserves the total number of elements.

Two details differ from the single-head case. First, the scale factor is $1/\sqrt{hd}$ rather than $1/\sqrt{e}$: within each head the key–query dot products grow with the per-head width hd, not with the full embedding e. Second, softmax is applied along dimension 3 (the key-position axis in the layout $[b, h, s, s]$) rather than dimension 2, and the softmax backward uses $\mathit{sumDim} \mathbin{@} 3$ and a broadcast reshape to $[b, h, s, 1]$ accordingly.

The backward pass is structurally identical to the single-head case. Gradients for Q, K, and V are computed in the $[b, h, s, hd]$ layout and collapsed to $[b, s, e]$ via *mergeHeads* before being passed to *wGrad*. Because the projection matrices remain $[e, e]$ regardless of h, *wGrad* is shared unchanged between the two implementations.

Like every other layer in this chapter, both attention variants are *ParaLens′* values and compose into larger architectures via $(\bullet)$. The 4-tuple parameter type is absorbed automatically into the product type of the enclosing model by the composition rule, with no boilerplate required.

Chapter 5

Loss Functions

A neural network assembled from *ParaLens′* values computes a typed transformation from inputs to predictions, but it does not yet know *what* to optimise. Loss functions supply this missing piece. In the lens framework they are not a separate concept bolted on from the outside; they are simply further *ParaLens′* values that can be composed with the network in the ordinary way. This chapter describes their type, their role in the composed diagram, and the three concrete instances provided by the library.

5.1 Losses, Caps, and the Learning Rate

Every loss function in this framework has the type

$$\mathit{ParaLens'}\ (t\ \mathit{shape})\ (t\ \mathit{shape})\ (t\ [\,])$$

Reading the three type arguments: the *parameter* $p = t\ shape$ is the target tensor; the *input* $a = t\ shape$ is the network's prediction; and the *output* $b = t\ [\,]$ is a scalar—a zero-dimensional tensor carrying a single floating-point number.

The loss reduces the network's prediction to a scalar, but it does not yet close the diagram to the monoidal unit. Composing a network *net* with a loss *loss* via $(\bullet)$ gives a term whose output is still $t\ [\,]$, not the unit type:

$$\mathit{net} \bullet \mathit{loss}\ :\ \mathit{ParaLens'}\ (\mathit{net\_params}, t\ \mathit{shape})\ (t\ \mathit{input\_shape})\ (t\ [\,]).$$

The true *cap*—the morphism that closes the final wire to the monoidal unit $I = ()$—is the learning rate. The library formalises this as a type synonym:

$$\textbf{type}\ \mathit{LRLens}\ l\ l' = \mathit{Lens}\ l\ l'\ ()\ ()$$

LRLens $l\ l'$ has $()$ as *both* output types: the forward pass discards the scalar (outputting the unit), and the backward pass emits the learning rate. The general constructor is:

$$learningRate :: (l \to l') \to LRLens\ l\ l'$$
$$learningRate\ alpha = lens\ (const\ ())\ (const \circ alpha)$$

where $const \circ alpha$ ignores the upstream () and applies *alpha* to the current state l to produce the gradient seed l'. For a constant tensor learning rate, $lrSmoothT$ specialises this by wrapping a Haskell scalar in a rank-0 tensor and negating it:

$$lrSmoothT\ lr = learningRate\ (const\ (negate\ (scalarT\ lr)))$$

Composing with $(\circ)$ closes the diagram fully:

$$(withGradDesc\ net \bullet loss) \circ lrSmoothT\ lr$$
$$:: ParaLens'\ (net\_params, target)\ input\ ()$$

The output type is now () — the monoidal unit — so no information leaves the diagram in the forward direction. The backward pass flows $-lr$ back through every layer as the gradient seed, updating all weight tensors in a single traversal. The *withGradDesc* wrapper (Chapter 6) handles the CRDC natural addition $p \leftarrow p + \partial p$; the cap supplies the sign and magnitude.

5.2 The Gradient Scaling Convention

As established in Section 2.3, the training loop performs the CRDC natural addition

$$\theta_{\text{new}} \;=\; \theta_{\text{old}} + \partial\theta,$$

where $\partial\theta$ is emitted by the backward pass; its sign and magnitude are the sole determinants of whether the step is descent or ascent. The backward pass of the MSE loss (Section 5.3) produces

$$\partial p \;=\; \tfrac{2}{N}\,\alpha \cdot (p - t),$$

where t is the target, N is the total number of output elements, and *alpha* is the scalar b' received from above. With $\alpha = -\eta$ (a *negative* learning rate), the update becomes

$$p_{\text{new}} \;=\; p_{\text{old}} + \tfrac{2}{N}(-\eta)(p - t) \;=\; p_{\text{old}} - \tfrac{2\eta}{N}(p - t),$$

which is exact gradient descent on the mean-squared-error loss. The $\frac{2}{N}$ factor arises from differentiating $\frac{1}{N}\sum_i (p_i - t_i)^2$; without it the update would correspond to a different (unnormalised) loss, breaking equivalence with standard automatic differentiation frameworks. The negative sign is not a convention trick; it is the mechanism by which the additive

CRDC structure produces descent rather than ascent. Choosing $\alpha > 0$ gives ascent; the library includes *deepDreamLoss*—a dot-product loss $\langle t, p \rangle$ with symmetric gradients—to support activation-maximisation settings without any change to the training loop.

5.3 Mean-Squared Error: `lossSmooth`

The MSE loss is a *ParaLens′* over tensors of arbitrary *shape*:

```
lossSmooth
  :: ( TrivialFacts shape
     , T.BasicArithmeticDTypeIsValid dv dt
     , T.StandardFloatingPointDTypeValidation dv dt)
  ⇒ ParaLens' (t shape) (t shape) (t [])
```

The constraint alias *TrivialFacts shape* captures two hasktorch API requirements—

```
type TrivialFacts shape =
  (shape~T.Broadcast shape shape
  , shape~T.Reverse (T.Reverse shape))
```

—that are semantically trivial (every shape satisfies them) but must be supplied as explicit witnesses to satisfy the type-checker when calling $T.sub$ and $T.mul$.

The implementation is:

```
lossSmooth = lens fwd (flip rev')
  where
    fwd          = uncurry $ T.mseLoss @ T.ReduceMean
    rev' alpha = (id &&& T.neg) ∘ T.mul alpha ∘ uncurry T.sub
                   -- fans out gradient to both outputs: (d, -d)
```

The forward pass delegates to $T.mseLoss @ T.ReduceMean$, which computes $\frac{1}{n}\sum_i (t_i - p_i)^2$ averaged over all elements. The backward pass applies the chain rule: with N elements and incoming scalar α from the learning-rate cap,

$$\partial t = \tfrac{2}{N}\,\alpha\,(t - p), \qquad \partial p = -\tfrac{2}{N}\,\alpha\,(t - p).$$

5.4 Softmax Cross-Entropy: `softMaxCELoss`

For classification tasks the cross-entropy loss fuses the softmax into the loss for numerical stability:

```
softMaxCELoss
  :: (T.All KnownNat [x, c]
     , T.BasicArithmeticDTypeIsValid dv dt
     , T.StandardFloatingPointDTypeValidation dv dt
     , T.SumDTypeIsValid dv dt, T.MeanDTypeValidation dv dt)
  ⇒ ParaLens' (t [x, c]) (t [x, c]) (t [])
```

The parameter is a distribution over c classes for each of x examples (one-hot or soft labels); the input is the corresponding logit tensor.

```
softMaxCELoss = lens fwd rev
  where
    fwd (bt, bp) = negate ∘ T.meanAll ∘ T.sumDim @ 1
                   $ bt * T.logSoftmax @ 1 bp
    rev (bt, bp) d  = (T.mul d $ negate $ T.log q, T.mul d $ q − bt)
      where q       = T.softmax @ 1 bp
```

The forward pass computes

$$-\frac{1}{x}\sum_{i=1}^{x}\sum_{c=1}^{C} t_{ic}\log q_{ic}, \qquad q = \mathrm{softmax}(p),$$

the standard categorical cross-entropy averaged over the batch. Throughout, the @1 type application selects axis 1—the class dimension—so that softmax and the summation operate over classes while leaving axis 0 (the batch) intact.

The backward pass exploits the well-known simplification that arises when softmax and cross-entropy are fused: the gradient with respect to the logits is simply $d\,(q - t)$. The gradient with respect to the targets, $-d \log q$, is available because the lens type treats both arguments symmetrically; in practice the training loop discards it, but it is useful in meta-learning settings where the targets themselves are learnable.

Chapter 6

Optimisers

A layer such as *matMulLensCore* knows how to compute gradients with respect to its weights but not what to *do* with them. The gradient update rule—gradient descent, momentum, Adam—is a separate concern, and the framework keeps it separate. Optimisers are lenses plugged into the parameter slot of a layer via *repara* (Section 3.7), converting raw gradients into updated parameters without touching the layer's forward or backward pass.

6.1 Gradient Descent

The simplest update rule is plain gradient descent: add the incoming gradient directly to the current parameter. In a cartesian reverse differential category this is the *natural addition* $\theta \leftarrow \theta + \partial\theta$; the sign of $\partial\theta$ (controlled by the learning-rate cap) determines whether the step is descent or ascent. The implementation is a single line:

$$
\begin{aligned}
&\mathit{gradUpdate} :: \mathit{Num}\ p \Rightarrow \mathit{Lens'}\ p\ p \\
&\mathit{gradUpdate} = \mathit{lens}\ \mathit{id}\ (+)
\end{aligned}
$$

The getter *id* reads the parameter unchanged; the setter $(+)$ adds the incoming gradient p' to the stored parameter p, returning $p + p'$. Equipping any layer with this rule is a one-liner:

$$
\begin{aligned}
&\mathit{withGradDesc} :: \mathit{Num}\ p \Rightarrow \mathit{ParaLens}\ p\ p\ a\ a'\ b\ b' \rightarrow \mathit{ParaLens}\ p\ p\ a\ a'\ b\ b' \\
&\mathit{withGradDesc} = \mathit{repara}\ \mathit{gradUpdate}
\end{aligned}
$$

The full gradient-descent training step is then:

$$
\begin{aligned}
&(\mathit{withGradDesc}\ \mathit{net} \bullet \mathit{loss}) \circ \mathit{lrSmoothT}\ \mathit{lr} \\
&\quad :: \mathit{ParaLens'}\ (\mathit{net\_params}, \mathit{target})\ \mathit{input}\ ()
\end{aligned}
$$

withGradDesc ensures each backward pass computes $\theta \leftarrow \theta + \partial\theta$; *lrSmoothT lr* (Section 5.1) provides the negative seed $\partial\theta = -\eta \cdot \partial L/\partial\theta$ that makes the step descent. Neither component knows about the other.

6.2 Momentum

Gradient descent converges slowly in the presence of high curvature. Momentum methods accumulate a velocity term that damps oscillations and accelerates convergence along low-curvature directions. In the lens framework, a momentum optimiser is a *Lens′* that *expands* the parameter type from *t shape* to (*t shape*, *t shape*), carrying the velocity buffer alongside the weights:

type *Momentum* = *Lens′* (*t shape*, *t shape*) (*t shape*)

The state (v, p) flows on the vertical parameter wire: p is the weight tensor read by the layer, and v is the velocity buffer memorised between steps.

SGD with Momentum

momentum :: *Scalar a* ⇒ *a* → *Momentum*
momentum = *lens snd* ∘ *momrev*

The getter *snd* extracts the current weights p for the layer. The setter *momrev gamma* (v, p) p' computes:

$$v' = -\gamma v + \partial p, \qquad p_{\text{new}} = p + v',$$

where ∂p is the incoming gradient. Setting $\partial p = -\eta\, \partial L/\partial p$ (from *lrSmoothT*) and expanding gives the standard SGD-with-momentum update. The velocity decays by γ each step and accumulates the gradient.

Nesterov Momentum

nesterov :: *Scalar a* ⇒ *a* → *Momentum*
nesterov gamma = *lens* (*uncurry fwd*) (*momrev gamma*)
  where *fwd v p* = *p* + *T.mulScalar gamma v*

The only difference from plain momentum is the getter: instead of returning p it returns the lookahead position $p + \gamma v$, so the gradient is evaluated at the next predicted iterate. The setter *momrev gamma* is unchanged. Both *momentum* and *nesterov* plug into any layer via *repara* (*momentum* 0.9) *layer*, which expands the parameter type from p to (v, p) with the velocity buffer initialised automatically.

6.3 Adaptive Methods

Adaptive methods maintain per-parameter statistics and scale the effective learning rate accordingly. They are again *Lens′* values that expand the parameter type.

AdaGrad

$$
\begin{aligned}
&\mathit{adaGrad} :: (\mathit{Scalar}\ a, \mathit{Fractional}\ a) \Rightarrow a \to \mathit{Momentum}\\
&\mathit{adaGrad}\ \mathit{eps} = \mathit{lens}\ \mathit{snd}\ \mathit{rev}\\
&\quad \textbf{where}\\
&\qquad \mathit{rev}\ (g, p)\ p' = (g', p + \mathit{update} * p')\\
&\qquad\quad \textbf{where}\\
&\qquad\qquad g' \ = g + p' * p'\\
&\qquad\qquad \mathit{update} = T.\mathit{mulScalar}\ \mathit{eps} \circ T.\mathit{reciprocal}\\
&\qquad\qquad\quad \circ\ T.\mathit{addScalar}\ \mathit{delta}\ \$\ T.\mathit{sqrt}\ g'
\end{aligned}
$$

The accumulator g stores the running sum of squared gradients. The per-parameter effective learning rate is $\varepsilon/\sqrt{g' + \delta}$, which shrinks automatically for parameters that receive large or frequent updates, and stays large for infrequently-updated parameters. The constant $\delta = 10^{-7}$ prevents division by zero.

Adam

Adam maintains two exponentially decaying moment estimates:

$$
\begin{aligned}
&\mathit{adam} :: (\mathit{Scalar}\ a, \mathit{Fractional}\ a) \Rightarrow a \to a \to a \to \mathit{Momentum2}\\
&\mathit{adam}\ \mathit{beta1}\ \mathit{beta2}\ \mathit{eps} = \mathit{lens}\ \mathit{snd}\ \mathit{rev}\\
&\quad \textbf{where}\\
&\qquad \mathit{rev}\ ((m, v), p)\ p' = ((m', v'), p + T.\mathit{mulScalar}\ \mathit{eps}\ \mathit{update})\\
&\qquad\quad \textbf{where}\\
&\qquad\qquad m' = T.\mathit{mulScalar}\ \mathit{beta1}\ m + T.\mathit{mulScalar}\ (1 - \mathit{beta1})\ p'\\
&\qquad\qquad v' \ = T.\mathit{mulScalar}\ \mathit{beta2}\ v + T.\mathit{mulScalar}\ (1 - \mathit{beta2})\ (p' * p')\\
&\qquad\qquad \mathit{update} = m'\ /\ T.\mathit{addScalar}\ \mathit{delta}\ (T.\mathit{sqrt}\ v')
\end{aligned}
$$

where $\mathit{Momentum2} = \mathit{Lens}'\ ((t\ \mathit{shape}, t\ \mathit{shape}), t\ \mathit{shape})\ (t\ \mathit{shape})$ carries state $((m, v), p)$. m is the first moment (exponential moving average of gradients), v is the second moment (exponential moving average of squared gradients), and ε is the step size. The update $m'/\sqrt{v' + \delta}$ normalises the gradient by its recent root-mean-square, reducing the effective learning rate for high-variance parameters.

Bias correction. This implementation omits the bias-correction factors from the original Adam paper [18], using the raw moments m' and v' directly. The effect is negligible once training is established; including correction would require threading a step counter through the optimiser state, adding a type parameter beyond what the thesis examples require—and beyond the original Cruttwell et al. framework [4].

The deepened state type $((m, v), p)$ vs *Momentum*'s (v, p) is automatic: applying *repara* (*adam* 0.9 0.999 1e−3) expands the parameter type to $((m, v), \mathit{weights})$ with no changes to the layer or the rest of the network.

6.4 Per-layer Optimisers

One key payoff of the *repara* mechanism is that different layers can use *different optimisers* with no coordination between them:

$\mathit{net} = \mathit{repara}\ (\mathit{momentum}\ 0.9)\ \mathit{matMulLensCore}$
- $\mathit{repara}\ (\mathit{adaGrad}\ 1\text{e}{-2})\ \mathit{matMulLensCore}$

The combined parameter type is $((v, p_1), (g, p_2))$—each layer carries its own independent optimiser state, tracked automatically through the product type from $(\bullet)$. No global optimiser object is needed; the type system enforces that each layer's update rule stays local.

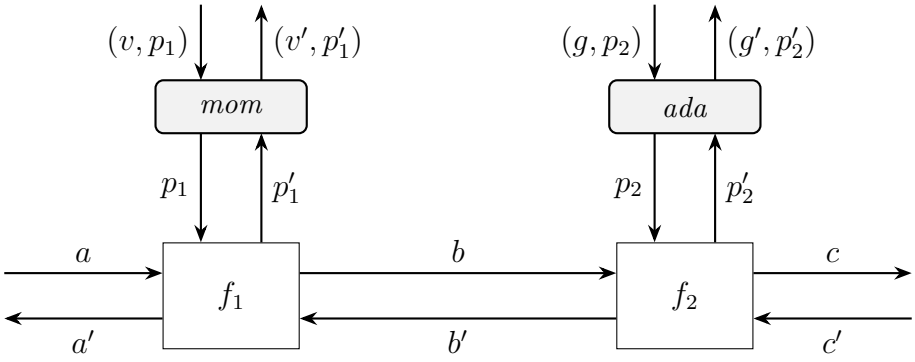

Figure 6.1: Two layers with independent per-layer optimisers. Layer f_1 uses SGD with momentum (state (v, p_1)); layer f_2 uses AdaGrad (state (g, p_2)). Data flows forward on the upper wires ($a \to b \to c$) and backward on the lower wires ($c' \to b' \to a'$); the two parameter channels are structurally independent, carrying different state types on the vertical axis.

This contrasts with conventional frameworks such as PyTorch or JAX, where the optimiser is a global object that must be explicitly configured with which parameters belong to which update rule, and where mixing rules across layers requires manual parameter grouping. Here the types enforce the separation: the outer parameter type $((v, p_1), (g, p_2))$ is the product of two independent optimiser states, and the composition rule $(\bullet)$ assembles it automatically from the individual *repara* calls.

Chapter 7

Case Studies and Evaluation

The preceding chapters described the framework's abstractions in isolation. This chapter grounds them in four concrete applications. The first, Iris flower classification, is a standard multi-layer perceptron benchmark small enough to inspect the output of GHC's optimiser directly. The second, MNIST digit recognition, shows the same compositional style scaling to a convolutional architecture. The third demonstrates that the framework is not limited to discriminative tasks: an MNIST autoencoder is expressed as two ordinary *ParaLens′* pipelines composed end-to-end, with the bottleneck constraint enforced at the type level. The fourth revisits MNIST with a residual architecture, demonstrating that *skipPara* composes into larger models without modification and that its parameter type is inferred automatically by the type system. Together they demonstrate that the lens abstraction adds zero runtime cost and that the type system catches architectural mistakes at compile time rather than at runtime.

7.1 Iris Flower Classification

The Iris dataset comprises 150 specimens of three species of iris, each described by four real-valued morphological measurements (sepal and petal length and width). The classification task is a standard MLP benchmark and is used here specifically because the network is small enough to read GHC Core output in full and compare it line for line against a hand-written implementation.

Model Definition

The architecture is n hidden fully-connected layers of width 4, followed by a single output layer that maps 4 features to 3 class logits. Three type aliases encode this at the type level:

$$\textbf{type } \mathit{InnerLayer}\ dv\ dt = \mathit{MMP}\ dv\ dt\ 4\ 4$$
$$\textbf{type } \mathit{IParams}\ n\ dv\ dt\ = (\mathit{StackedN}\ n\ (\mathit{InnerLayer}\ dv\ dt), \mathit{MMP}\ dv\ dt\ 3\ 4)$$

$\mathit{MMP}\ dv\ dt\ out\ \textbf{in} = (T.\mathit{Tensor}\ dv\ dt\ [out, \textbf{in}\ ], T.\mathit{Tensor}\ dv\ dt\ [out])$ is a weight matrix paired with a bias vector (Chapter 4). *InnerLayer* is a $4 \to 4$ affine map. $\mathit{IParams}\ n\ dv\ dt$

is the product of n inner layers followed by one output layer; for $n = 2$ it expands to $((MMP\ dv\ dt\ 4\ 4, MMP\ dv\ dt\ 4\ 4), MMP\ dv\ dt\ 3\ 4)$—a fully concrete nested tuple whose structure mirrors the layer order.

The *ParaLens′* model is:

$$
\begin{aligned}
&nMuls :: (CanStack\ n, SaneDT\ dv\ dt) \Rightarrow \\
&\qquad ParaLens'\ (StackedN\ n\ (MMP\ dv\ dt\ m\ m)) \\
&\qquad\qquad (T.Tensor\ dv\ dt\ [b, m]) \\
&\qquad\qquad (T.Tensor\ dv\ dt\ [b, m]) \\
&nMuls = stackN\ @\ n\ (matMulLens \circ sigmoid) \\
&irisModel :: (CanStack\ n, SaneDT\ dv\ dt) \Rightarrow \\
&\qquad ParaLens'\ (Inp\ (T.Tensor\ dv\ dt\ [b, 4]), IParams\ n\ dv\ dt) \\
&\qquad\qquad () \\
&\qquad\qquad (Out\ (T.Tensor\ dv\ dt\ [b, 3])) \\
&irisModel = argToPara \bullet nMuls\ @\ n \bullet matMulLens \circ sigmoid
\end{aligned}
$$

argToPara (Section 3.7) converts the raw input tensor into a *ParaLens′* with a unit parameter, making it composable with $(\bullet)$. Each $(\bullet)$ join adds one more layer, automatically extending the parameter type. The final $matMulLens \circ sigmoid$ is the output layer—relu is parameter-free so it composes with plain $(\circ)$ rather than with $(\bullet)$, leaving the output layer's parameter type clean.

The hand-rolled forward pass for $n = 1$ is:

$$
\begin{aligned}
&handRolledTest\ (i, (mb, mb')) = layer\ mb' \circ layer\ mb\ \$\ i \\
&\quad \textbf{where} \\
&\qquad layer\ (m, b) = T.sigmoid \circ T.add\ b \circ (`T.matmul`transp\ m)
\end{aligned}
$$

The two definitions produce identical GHC Core, as shown in the next subsection.

Zero-Overhead Abstraction

The elimination happens in two stages. First, the van Laarhoven representation

$$Lens\ s\ t\ a\ b\ =\ \forall f.\ Functor\ f \Rightarrow (a \to f\ b) \to s \to f\ t$$

is specialised at the call site. The forward pass (*runFullModel*) instantiates $f = \texttt{Const}\ b$, for which *fmap* = *flip const*, collapsing every lens in the chain to its getter. The backward pass instantiates $f = \texttt{Identity}$, for which *fmap* = *coerce*, collapsing to the setter–getter chain. In both cases the $\forall\ f$ disappears and all intermediate *fmap* calls and $(,)$ wrappers

reduce to their concrete definitions. Second, the {-# INLINE #-} pragmas on (•), *repara*, *stackN*, and their auxiliaries (*swapFst*, *rotate*, *leftLens*, *rightLens*) allow GHC to discharge the remaining composition layers, leaving only the raw tensor operations.

Compiling both *test* = *runFullModel* (*irisModel* @ 1) and *handRolledTest* with `-O2` and `-ddump-simpl` produces worker functions `$wtest` and `$whandRolledTest` that are structurally identical.

Each worker accepts nine unboxed arguments—a `ForeignPtr` pair for the input tensor and two pointer pairs per weight/bias group—and performs the same eight operations in the same order:

```
$s$wtranspose  w1_ptr              --  W1^T
$wmatmul_tt    x_ptr    W1^T       --  X * W1^T
$wadd          b1_ptr   (X*W1^T)   --  + b1
$wsigmoid_t    (X*W1^T + b1)       --  sigma_1
$s$wtranspose  w2_ptr              --  W2^T
$wmatmul_tt    h1_ptr   W2^T       --  h1 * W2^T
$wadd          b2_ptr   (h1*W2^T)  --  + b2
$wsigmoid      (h1*W2^T + b2)      --  sigma_2
```

The wrapper functions also match: both `test1` and `handRolledTest` perform twelve nested `case` analyses to unpack the input tuple into nine raw pointer arguments, then tail-call their respective workers. The only differences in the entire output are internal variable names and coercion-tag numbers (`Co:42` vs. `Co:960` for the final cast), which are GHC-internal type-representation artefacts with no runtime cost.

The lens machinery—*Lens* (p, a), (•), *alongside*, *swapFst*, *rotate*, the $(,)$ tuple constructors—is entirely absent from the optimised output. The user writes four lines of lens composition; GHC emits the same sequence of LibTorch FFI calls as the hand-written version.

Type-Level Layer Depth

The depth n is a compile-time natural number, not a runtime value. `Stack.hs` converts it to a Peano numeral and uses typeclass induction to build both the parameter type and the (•) composition tree simultaneously:

data *PeanoNat* = *One* | *Succ PeanoNat*

type *family Stacked* (n :: *PeanoNat*) (m :: *Type*) **where**
  Stacked One m = m
  Stacked (*Succ* n) m = $(m, \mathit{Stacked}\ n\ m)$

```
class StackN (n :: PeanoNat) where
  stack :: ParaLens' p a a → ParaLens' (Stacked n p) a a
instance StackN One where
  stack l = l
instance StackN n ⇒ StackN (Succ n) where
  stack l = l • stack @ n l
```

The public API bridges GHC's built-in *Nat* literals:

```
type StackedN (n :: Nat) m = Stacked (ToPeano n) m
type CanStack n            = StackN (ToPeano n)
stackN :: CanStack n ⇒ ParaLens' p a a → ParaLens' (StackedN n p) a a
stackN = stack @ (ToPeano n)
```

For $n = 3$, *StackedN* 3 (*MMP* . .) resolves at compile time to (*MMP* . . , (*MMP* . . , *MMP* . .)). Crucially, GHC always sees a concrete product type, not a list or a vector. This is precisely what enables the zero-overhead property from the previous subsection: because the parameter type is a fully known nested tuple at each n, the inliner can fully specialise and unbox the entire parameter structure, producing the same Core as a hand-written n-layer network at any fixed depth.

Adding or removing a hidden layer is a one-character change to the type application @n; no other code changes.

The same typeclass induction extends to initialisation. A *RandInit* instance on pairs lifts random sampling to any product type automatically:

```
instance (RandInit l, RandInit r) ⇒ RandInit (l, r) where
  randInit = liftA2 (,) randInit randInit
```

irisInitParams@n therefore generates a freshly randomised *IParams n dv dt* of the correct depth with no boilerplate.

Devices, Dtypes, and Static Shapes

The tensor type *T.Tensor dv dt shape* carries three compile-time parameters: the device *dv*, the element dtype *dt*, and the shape *shape* (a type-level list of dimension sizes). The *SaneDT* constraint alias bundles the training requirements:

```
type SaneDT dv dt =
  ( T.KnownDType dt
```

, $T.StandardFloatingPointDTypeValidation\ dv\ dt$
, $CanMMLens\ dv\ dt$)

$T.StandardFloatingPointDTypeValidation\ dv\ dt$ is unsatisfied by integer or boolean dtypes, so any attempt to train with a non-differentiable dtype is rejected by the type checker before a single line of training code runs. Switching between precisions or between CPU and GPU requires only a change to the type application at the call site:

$irisGetEpoch \mathbin{@} 1 \mathbin{@} (T.CPU, 0) \mathbin{@} T.Double$ -- CPU, double precision
$irisGetEpoch \mathbin{@} 1 \mathbin{@} (T.CUDA, 0) \mathbin{@} T.Float$ -- GPU, single precision

The model definition, training loop, and loss function are entirely unchanged. Shape mismatches—for example, feeding a $[batch, 10]$ tensor to a layer expecting $[batch, 4]$—are type errors caught by the shape in $T.Tensor\ dv\ dt\ [b, 4]$. No runtime assertions or dynamic shape checks are needed anywhere in the framework.

The small size of the Iris model means that the performance differences between Float and Double are negligible; Table 7.1 in Section 7.2 gives a more realistic benchmark on the larger MNIST dataset.

7.2 MNIST Digit Recognition

MNIST is a dataset of 70,000 greyscale images of handwritten digits (0–9) at 28×28 pixels, split into 60,000 training and 10,000 test examples. The model used here is a small convolutional network followed by a dense output layer:

```
[batch,  1, 28, 28]
  conv1 (3x3, 3 filters)  -> relu -> maxpool(2x2)  -> [batch,  3, 13, 13]
  conv2 (4x4, 5 filters)  -> relu -> maxpool(2x2)  -> [batch,  5,  5,  5]
  flatten                                           -> [batch, 125]
  dense (125 -> 10)                                 -> [batch,  10]
```

The 1,527-parameter model (conv1: 27, conv2: 240, dense: 1,260) is intentionally modest; the purpose is to demonstrate the framework's compositional style on a multi-stage architecture, not to achieve state-of-the-art accuracy.

Performance

Parameter Types

Four type aliases encode the architecture at the type level:

Precision	Mean (ms)	Min (ms)	Max (ms)	Std dev (ms)	R^2
T.Float (32-bit)	251	232	260	16	0.993
T.Double (64-bit)	289	284	292	5	1.000

Table 7.1: MNIST CNN training: wall-clock time per epoch by floating-point precision, CPU, batch size 32, 6,000 training examples. Measured with Criterion after a full-epoch warmup pass for each dtype; min/max are the 95% confidence bounds on the mean. Float is 15% faster, consistent with wider SIMD throughput for 32-bit arithmetic.

$$\textbf{type } \mathit{Conv1K}\ \mathit{dev}\ \mathit{dt} = \mathit{T.Tensor}\ \mathit{dev}\ \mathit{dt}\ [3,1,3,3]$$
$$\textbf{type } \mathit{Conv2K}\ \mathit{dev}\ \mathit{dt} = \mathit{T.Tensor}\ \mathit{dev}\ \mathit{dt}\ [5,3,4,4]$$
$$\textbf{type } \mathit{DenseP}\ \mathit{dev}\ \mathit{dt} = \mathit{MMP}\ \mathit{dev}\ \mathit{dt}\ 10\ 125$$
$$\textbf{type } \mathit{MnistP}\ \mathit{dev}\ \mathit{dt} = (\mathit{Conv1K}\ \mathit{dev}\ \mathit{dt}, (\mathit{Conv2K}\ \mathit{dev}\ \mathit{dt}, \mathit{DenseP}\ \mathit{dev}\ \mathit{dt}))$$

MnistP is the product of all learnable parameters in layer order. A kernel of the wrong shape—say, $[3,1,4,4]$ instead of $[3,1,3,3]$ for *Conv1K*—is a compile-time type error. The product structure is assembled automatically by $(\bullet)$: each composition step extends the parameter tuple by one more layer's worth of weights, so *MnistP* is inferred rather than written by hand.

The *SaneMnist* constraint alias bundles every capability required for training, including dtype validity for mean reduction (used by the loss) and comparison operations (used by the accuracy metric):

$$\begin{aligned}&\textbf{type } \mathit{SaneMnist}\ \mathit{dev}\ \mathit{dt} =\\ &\quad (\mathit{CanMMLens}\ \mathit{dev}\ \mathit{dt}\\ &\quad , \mathit{T.StandardFloatingPointDTypeValidation}\ \mathit{dev}\ \mathit{dt}\\ &\quad , \mathit{T.MeanDTypeValidation}\ \mathit{dev}\ \mathit{dt}\\ &\quad , \mathit{T.KnownDevice}\ \mathit{dev}\\ &\quad , \mathit{T.KnownDType}\ \mathit{dt})\end{aligned}$$

Any device/dtype combination that fails to satisfy the full set—for example, a device with no mean-reduction support—is rejected at compile time.

Model Composition

$$\mathit{mnistModel} = \mathit{argToPara}$$
- $\mathit{withGradDesc}\ \mathit{convLens} \circ \mathit{relu} \circ \mathit{maxPool} \mathbin{@} (2,2) \mathbin{@} (2,2) \mathbin{@} (0,0)$
- $\mathit{withGradDesc}\ \mathit{convLens} \circ \mathit{relu} \circ \mathit{maxPool} \mathbin{@} (2,2) \mathbin{@} (2,2) \mathbin{@} (0,0)$
- $\mathit{rightLens}\ (\mathit{flatten} \mathbin{@} b \mathbin{@} [5,5,5]) \circ \mathit{matMulLens}$

Each (•) arm contributes one block to both the forward pass and the combined parameter type. Within each convolutional arm, *relu* and *maxPool*@... are parameter-free lenses composed with plain (∘) rather than (•), so they thread data through without extending the parameter type. The type applications to *maxPool*— kernel size, stride, and padding—are static; the output shape after each pooling step is checked at compile time. *withGradDesc convLens* (Section 6.1) wraps each convolutional layer with the CRDC natural addition, so the gradient update $\theta \leftarrow \theta + \partial\theta$ is baked into the layer itself.

The final arm uses *rightLens* to pre-process the data component of the $(param, \lambda, \mathbf{data})$ pair before the dense layer:

$$(MMP,\ [batch, 5, 5, 5]) \xrightarrow{\texttt{rightLens (flatten)}} (MMP,\ [batch, 125]) \xrightarrow{\texttt{matMulLens}} [batch, 10].$$

flatten is a pure Iso' with no learnable parameters and no effect on the gradient path through *MMP*. Composing it via *rightLens* (which lifts a lens on the second component of a pair) keeps the reshape and the matmul as a single $ParaLens'$ unit, avoiding any intermediate allocation.

He Initialisation

```
mnistInitParams :: IO (MnistP dev dt)
mnistInitParams = do
  let sc x    =    T.mulScalar (x :: Float)
  c1 ← sc (sqrt (2 / 9))    ⟨$⟩ T.randn
  c2 ← sc (sqrt (2 / 48))   ⟨$⟩ T.randn
  w  ← sc (sqrt (2 / 125)) ⟨$⟩ T.randn
  let b = T.zeros
  pure (c1, (c2, (w, b)))
```

He initialisation [19] sets each layer's weight standard deviation to $\sqrt{2/\text{fan\_in}}$, keeping the variance of activations constant across ReLU layers and preventing vanishing gradients in early training. The fan-in values are $1 \times 3 \times 3 = 9$ for *Conv1K*, $3 \times 4 \times 4 = 48$ for *Conv2K*, and 125 for the dense weight matrix. The bias is initialised to zero.

Loss and Training

The loss and training models follow the same pattern as Iris:

```
mnistModelLoss = mnistModel • softMaxCELoss
mnistModel'    = mnistModel • softMaxCELoss ∘ lrSmooth 1e−4
```

softMaxCELoss (Section 5.4) fuses the softmax and log operations for numerical stability. *lrSmooth* 1e−4 caps the output wire to the unit type, seeding backpropagation with -10^{-4}.

One training epoch is a strict left fold over all batches:

$$mnistEpoch\ trainT\ params = trainMany\ mnistModel'\ params\ trainT$$

Inference extracts predictions by taking the argmax along the class axis:

$$\begin{aligned} &mnistPredict\ p\ x = \\ &\quad U.asValue \circ T.toDynamic\ \$ \\ &\qquad T.argmax\ @\ 1\ @\ T.DropDim\ (runFullModel\ mnistModel\ (x, p)) \end{aligned}$$

The @1 selects the class dimension and $@T.DropDim$ removes it, returning a list of integer digit labels. The type of $runFullModel\ mnistModel\ (x, p)$ is $T.Tensor\ dev\ dt\ [b, 10]$, so the argmax and the label extraction are statically typed throughout.

Training Results

The model was trained for 400 epochs on a 6,000-example subset of the MNIST training set (10%)—chosen to keep each epoch fast enough to run on a CPU-only machine—with batch size 32 and learning rate 10^{-4}. Test accuracy was evaluated on the full 10,000 example test set after each epoch. The model reaches approximately **90% test accuracy** around epoch 300, peaking at 90.4% at epoch 299 and plateauing thereafter. Given that the model has only 1,527 parameters and is trained on one-tenth of the available data, the result confirms that the framework's compositional forward and backward passes are numerically correct end-to-end and that the lens abstraction imposes no obstacle to learning.

7.3 MNIST Autoencoder

Autoencoders demonstrate that the framework is not limited to discriminative tasks. An autoencoder learns to compress its input into a low-dimensional latent representation and then reconstruct the original from that representation, trained by minimising reconstruction error rather than classification loss. In the parametric lens framework, encoder and decoder are each ordinary *ParaLens′* pipelines; composing them end-to-end with (•) produces the full model with no special casing.

Architecture

The model maps flattened MNIST images ($28 \times 28 = 784$ pixels) to a 32-dimensional latent space and back:

```
encoder: [b, 784] -> Dense(784->128) -> ReLU -> Dense(128->32)
decoder: [b,  32] -> Dense(32->128)  -> ReLU -> Dense(128->784) -> Sigmoid
```

The bottleneck—latent dimension 32 strictly smaller than input dimension 784—is enforced at the type level: the encoder output type $T.Tensor\ dv\ dt\ [b, LatentDim]$ must unify with the decoder input type, so a dimension mismatch is a compile-time error rather than a silent shape broadcast.

Parameter Types

Three type aliases capture the parameter structure:

$$\begin{aligned}
&\textbf{type}\ EncoderP\ dev\ dt = (MMP\ dev\ dt\ HiddenDim\ InputDim,\\
&\qquad\qquad\qquad\qquad MMP\ dev\ dt\ LatentDim\ HiddenDim)\\
&\textbf{type}\ DecoderP\ dev\ dt = (MMP\ dev\ dt\ HiddenDim\ LatentDim,\\
&\qquad\qquad\qquad\qquad MMP\ dev\ dt\ InputDim\ HiddenDim)\\
&\textbf{type}\ AEP\ dev\ dt \qquad = (EncoderP\ dev\ dt, DecoderP\ dev\ dt)
\end{aligned}$$

where $InputDim = 784$, $HiddenDim = 128$, and $LatentDim = 32$. $AEP\ dev\ dt$ is a nested product of four weight-bias pairs. As with the discriminative models, this type is not written by hand: it is assembled automatically by $(\bullet)$ from the encoder and decoder parameter types.

Model Composition

Encoder and decoder are each plain $ParaLens'$ values:

$$\begin{aligned}
encoderCore\ &::\ (SaneAE\ dev\ dt, KnownNat\ b)\\
&\Rightarrow ParaLens'\ (EncoderP\ dev\ dt)\\
&\qquad\qquad (T.Tensor\ dev\ dt\ [b, InputDim])\\
&\qquad\qquad (T.Tensor\ dev\ dt\ [b, LatentDim])\\
encoderCore\ &=\ matMulLens \circ relu \bullet matMulLens\\
decoderCore\ &::\ (SaneAE\ dev\ dt, KnownNat\ b)\\
&\Rightarrow ParaLens'\ (DecoderP\ dev\ dt)\\
&\qquad\qquad (T.Tensor\ dev\ dt\ [b, LatentDim])\\
&\qquad\qquad (T.Tensor\ dev\ dt\ [b, InputDim])\\
decoderCore\ &=\ matMulLens \circ relu \bullet matMulLens \circ sigmoid
\end{aligned}$$

The full autoencoder is a single further composition:

$$autoencoderModel = argToPara \bullet encoderCore \bullet decoderCore$$

Three (•) calls assemble the complete pipeline. The combined parameter type $(T.Tensor\ dev\ dt\ [b, InputDim], AEP\ dev\ dt)$ is inferred without annotation. The composition rule pairs each sub-lens's parameter wire into the product type automatically, in exactly the same way as the discriminative models of the preceding sections.

Loss and Training

Reconstruction quality is measured by mean-squared error. The trainable model is:

$$autoencoderModel' = autoencoderModel \bullet lossSmooth \circ lrSmooth\ 1e{-}3$$

lossSmooth (Section 5.3) replaces the cross-entropy loss of earlier models; no other part of the training infrastructure changes. Each training step presents a batch x of flattened images paired with itself as the reconstruction target:

$$aeEpoch\ trainT = flip\ (trainMany\ (autoencoderModel' \mathbin{@} BatchSize))\ trainT$$

where *trainT* is a list of (x, x) pairs. The parameter update, gradient flow, and epoch structure are handled identically to the discriminative case. The only difference visible at the call site is the loss: *lossSmooth* computes $\frac{1}{N}\sum_i (p_i - t_i)^2$ rather than cross-entropy, and the gradient seed flows backward through the decoder and into the encoder without any additional wiring.

7.4 Residual MNIST

The MNIST task is revisited with a residual architecture to show that *skipPara* (Section 4.5) integrates into a larger pipeline without modification. The model replaces the convolutional stack with a sequence of dense residual blocks, keeping the setting comparable to the plain MNIST model.

Parameter Types

Each residual block wraps two affine layers of the same width, so its parameter type is a pair of weight-bias pairs:

type $Hidden = 128$
type $NumBlocks = 3$
type $ResBlockP\ dev\ dt = (MMP\ dev\ dt\ Hidden\ Hidden, MMP\ dev\ dt\ Hidden\ Hidden)$
type $ResMnistP\ dev\ dt =$
$(MMP\ dev\ dt\ Hidden\ 784$
$, (StackedN\ NumBlocks\ (ResBlockP\ dev\ dt), MMP\ dev\ dt\ 10\ Hidden))$

ResMnistP is the product of an input projection, three stacked block parameter pairs, and an output layer, assembled automatically by the (•) composition rule.

Model Composition

A single residual block is one application of *skipPara*:

$$
\begin{aligned}
&\mathit{resBlock} :: \mathit{SaneRes}\ b\ \mathit{dev}\ \mathit{dt} \\
&\qquad \Rightarrow \mathit{ParaLens}'\ (\mathit{ResBlockP}\ \mathit{dev}\ \mathit{dt}) \\
&\qquad\qquad\qquad (\mathit{Tensor}\ \mathit{dev}\ \mathit{dt}\ [b, \mathit{Hidden}]) \\
&\qquad\qquad\qquad (\mathit{Tensor}\ \mathit{dev}\ \mathit{dt}\ [b, \mathit{Hidden}]) \\
&\mathit{resBlock} = \mathit{skipPara}\ (\mathit{matMulLens} \circ \mathit{relu} \bullet \mathit{matMulLens} \circ \mathit{relu})
\end{aligned}
$$

The parameter type (*ResBlockP dev dt*) is inherited from the inner pipeline; *skipPara* contributes no additional parameters. The full model stacks *NumBlocks* such blocks between a flattening input projection and a linear output layer:

$$
\begin{aligned}
&\mathit{resMnistModel} = \mathit{argToPara} \\
&\quad \bullet\ \mathit{rightLens}\ (\mathit{flatten} \mathbin{@} b \mathbin{@} [1, 28, 28]) \circ \mathit{matMulLens} \circ \mathit{relu} \\
&\quad \bullet\ \mathit{stackN} \mathbin{@} \mathit{NumBlocks}\ \mathit{resBlock} \\
&\quad \bullet\ \mathit{matMulLens}
\end{aligned}
$$

The pipeline is a drop-in replacement for *mnistModel*: the same training loop, loss combinator, and inference infrastructure apply unchanged.

He Initialisation

Weights use the same $\sqrt{2/\text{fan\_in}}$ rule as the MNIST model above, with fan-in 784 for the input projection and $\mathit{Hidden} = 128$ for every layer inside a residual block.

Longer-Range Skips

The architecture above chains three blocks with *stackN*, each carrying an independent *skipPara* connection spanning its own two layers—the standard residual block structure of He et al. [19]. The blocks are composed sequentially with no cross-block skip connections, matching the original ResNet design. For stages that change spatial resolution or channel count, the framework provides *projSkipPara* (Section 4.5), which runs a learned projection shortcut in parallel with f using the same *splitIso* fan-out and sum structure.

7.5 Performance Evaluation

Three implementations of the Iris two-layer MLP at depth $n = 1$ are compared using Criterion [20] on a CPU-only machine (GHC 9.8.4, LibTorch 2.9.1):

typed-hasktorch The *ParaLens* library with *Torch.Typed* tensors; forward and backward passes are pure Haskell lens composition, calling LibTorch only for primitive tensor operations.

hmatrix A second lens-based implementation wrapping LAPACK and BLAS via the HMatrix library [21]; the backward pass is also pure Haskell.

dynamic-hasktorch A baseline delegating both passes to PyTorch autograd via *runStep*, *flattenParameters*, and `.backward()`.

`raw_fwd` measures a single forward pass over one batch of eight samples; `epoch` measures one full training epoch—a strict left fold over all 18 mini-batches—including the backward pass and parameter update for every batch.

Benchmark	Implementation	Mean time	Rel. to typed
`raw_fwd`	`typed-hasktorch`	6.9 μs	1.00×
	`typed-hasktorch-handroll`	6.8 μs	0.99×
`epoch`	`hmatrix`	390 μs	0.57×
	`typed-hasktorch`	689 μs	1.00×
	`dynamic-hasktorch`	3,631 μs	5.27×

Table 7.2: Criterion benchmark results for the Iris MLP ($n = 1$, batch size 8, 18 batches per epoch, CPU). All times are wall-clock means; standard deviations were below 4% for all measurements.

The `raw_fwd` row confirms the GHC Core result of Section 7.1: the typed and hand-rolled implementations are statistically indistinguishable.

Variant	Mean time	Diff. from baseline	Overhead identified
`fold-foldM`	3,632 μs	—	baseline
`fold-foldl'`	3,649 μs	+17 μs (noise)	fold structure: ≈ 0
`forward-only`	251 μs	−3,381 μs	`.backward()` + update
`flattenParams-x18`	< 1 μs	—	Generic traversal: < 0.03 μs
Typed forward ×18		$18 \times 6.9 = 124$ μs	tensor arithmetic
`forward-only` − typed×18		$251 - 124 = 127$ μs	autograd graph build

Table 7.3: Overhead isolation for one dynamic epoch. The `forward-only` variant runs the full forward pass (including autograd graph construction) but never calls `.backward()`. Std devs: fold variants $< 1\%$, `forward-only` 4%.

Typed lens vs. dynamic autograd

The *ParaLens* training loop (689 μs) is **5.3× faster** than the dynamic autograd baseline (3,631 μs). Table 7.3 isolates each proposed source of overhead.

PyTorch backward pass: 93% of the cost. The `forward-only` variant runs the full forward pass for every mini-batch—including autograd graph construction, since the model parameters carry *requires_grad* = *True*—but never calls ∘*backward* (). It completes in 251 μs. The full epoch (3,631 μs) costs 3,381 μs more: **93%** of the total epoch time is spent inside PyTorch's C++ backward pass. This is the dominant cost of delegating differentiation to an external autograd engine. The *ParaLens* backward is a chain of Haskell closures that GHC inlines and optimises at compile time; at runtime it reduces to the same eight LibTorch FFI calls as the forward pass.

Autograd graph construction: 3.5%. Subtracting the typed forward cost ($18 \times 6.9 = 124$ μs) from the forward-only time (251 μs) leaves 127 μs attributable to PyTorch's tape construction: allocating *Node* objects, storing input pointers, and registering backward functions for each of the eight tensor operations in the two-layer network. Real but modest—under a quarter of the cost of the backward pass itself.

Fold structure and Generic traversal: negligible. Replacing *foldM* with an explicit *foldl′* chain changes the epoch time by 17 μs—within the noise floor. GHC compiles both to the same loop at `-O2`. Likewise, 18 calls to *flattenParameters*—the *Generic* traversal collecting the four model tensors into a list—complete in under 1 μs total ($< 0.03\%$ of the epoch).

Typed lens vs. HMatrix

The HMatrix baseline (390 μs) is **1.8× faster** than the typed lens implementation (689 μs) on Iris. HMatrix calls LAPACK and BLAS directly [21] and incurs lower per-call overhead than LibTorch for very small matrices: every LibTorch tensor operation passes through ATen's multi-device operator dispatch [22] before reaching the underlying BLAS kernel, a fixed cost that dominates for 4×4 matrices and is well-studied in the literature [23].

Both implementations perform the backward pass in pure Haskell; the per-call FFI cost is the sole driver of the gap. For larger matrices or batch sizes the typed lens implementation is expected to match HMatrix and exceed it via LibTorch's vectorised kernels.

7.6 Gradient Correctness Tests

The framework includes a test suite that cross-validates every primitive layer's forward and backward pass against PyTorch autograd, providing numerical evidence that the hand-derived gradients are correct.

Method

Each test constructs identical inputs in both the typed *ParaLens* implementation and a dynamic reference model, runs one gradient-descent step via *runStep* with learning rate 1, and recovers the autograd gradient as the difference between old and new parameters. The typed gradient is compared against this reference with a maximum absolute tolerance of 10^{-4}. Because the typed backward pass is statically compiled Haskell rather than a traced computation graph, any discrepancy would indicate an error in the hand-derived Jacobian, not a numerical accident.

Coverage

- *linear*: forward pass ($xW^\top$) and weight gradient ($\partial L/\partial W = grad^\top x$), batch size 1.
- *addLens*: forward pass (broadcast add) and bias gradient ($\partial L/\partial b = \sum_{\text{batch}} grad$), batch size 4.
- *sigmoid*: forward pass ($\sigma(x)$) and input gradient ($grad \cdot \sigma(x)(1-\sigma(x))$), 10 values.
- *relu*: forward pass and Heaviside input gradient, 10 values.
- *convLens*: forward pass (unit stride, zero padding), and kernel gradient via the *im2col* batched matrix multiply, on a $1 \times 1 \times 7 \times 7$ input with a $2 \times 1 \times 3 \times 3$ kernel.
- *flatten*: forward reshape and backward reshape (both are pure shape operations with no arithmetic, verified to be exact inverses).
- **Iris forward equivalence**: the full two-layer Iris model run on a batch of eight samples produces output identical to an equivalent dynamic hasktorch model given the same weights.
- **Iris epoch equivalence**: after one full training epoch (18 batches), the typed and dynamic models reach identical parameter values.

Relationship to Accuracy

Layer-level gradient tests are a stronger correctness signal than end-to-end accuracy: a model can converge to a non-trivial accuracy even with slightly incorrect gradients, whereas a Jacobian error above 10^{-4} would be caught directly. The epoch-equivalence test extends this to the composed training loop, confirming that composition via ($\bullet$) and the *repara* update rule interact correctly across an entire epoch on real data.

Chapter 8

Conclusion

8.1 Summary

This thesis has instantiated the categorical framework of Cruttwell et al. [4] as a working Haskell library for gradient-based machine learning. The central abstraction—that every component of a training pipeline is a parametric lens carrying a forward pass and a gradient pass as a single composable unit—is realised here with full static verification. Tensor shapes, mini-batch sizes, compute devices, and element dtypes are all encoded as type-level parameters, so a misconfigured architecture is rejected by the type checker rather than failing at runtime or producing a silent numerical error.

The library extends the original Python proof-of-concept in eight concrete directions. First, all implementations are written in batched form from the ground up: every layer, loss function, and optimiser operates over a leading batch dimension b that is a type-level `Nat`, whereas the original Cruttwell et al. implementation processes a single example at a time. Second, all tensor dimensions are static: GHC's `DataKinds` extension encodes every dimension as a type-level `Nat`, and type families compute output shapes for convolutions and pooling layers automatically, making spatial mismatches compile-time errors (Chapter 4). Third, device and dtype polymorphism are built into every model signature: a single *ParaLens′* definition is valid on CPU or CUDA and in 32-bit or 64-bit floating point, with constraint synonyms such as *SaneDT* preventing invalid device/dtype combinations from type-checking. Fourth, the framework is extended beyond the MLP scope of the original paper to autoencoder architectures: encoder and decoder are each ordinary *ParaLens′* pipelines that compose with $(\bullet)$ into a single end-to-end model, trained on MNIST images with MSE reconstruction loss. Fifth, scaled dot-product self-attention and its multi-head generalisation are implemented as *ParaLens′* values with fully manually derived backward passes—including the rank-one Jacobian correction for the softmax non-linearity—extending the framework to the attention mechanism that underlies modern transformer architectures (Chapter 4). Sixth, residual skip connections are realised as a single higher-order combinator *skipPara*, built entirely from existing lens primitives with

no new axioms; the identity gradient path that ensures gradients reach early layers even when the learned branch saturates emerges automatically from the combinator structure, requiring no separate backward-pass derivation (Section 4.5). Seventh, type-level Peano naturals and typeclass induction are used to build networks of variable depth n whose parameter type is a fully concrete nested tuple at each n, preserving GHC's ability to specialise and unbox the entire parameter structure and yielding zero overhead relative to a hand-written network of the same depth. Eighth, the zero-overhead property is verified directly by GHC Core inspection: compiling the lens-based forward pass and an equivalent hand-written forward pass with `-O2 -ddump-simpl` produces structurally identical worker functions, confirming that the entire *ParaLens* abstraction—composition, reparametrisation, the van Laarhoven $\forall$—is absent from the optimised output (Section 7.1).

8.2 Future Work

Several directions remain open for future investigation.

Recurrent architectures. Recurrent networks maintain hidden state across time steps, which does not fit the stateless composition pattern of $(\bullet)$ directly. One avenue is to treat the hidden state as an additional parameter, making the recurrence an instance of *repara* on the state wire. Whether this recovers standard BPTT or requires a categorical extension remains an open question.

Formal verification of lens laws. The lens laws (get-put, put-get, put-put) are satisfied structurally by construction in this framework, but are not machine-verified. Property-based testing with QuickCheck could confirm the laws hold for all implemented lenses under arbitrary inputs, providing stronger assurance than the informal argument from construction.

Discrete and probabilistic CRDCs. The Cruttwell et al. framework is defined for any CRDC, not just smooth Euclidean spaces. Instantiating the Haskell library for a discrete CRDC (e.g., a category of Boolean circuits) or a probabilistic one would generalise gradient-based learning beyond the real-valued setting and test the abstraction boundaries of the current design.

Bibliography

1. LeCun Y, Bengio Y, and Hinton G. Deep learning. Nature 2015; 521:436–44
2. Shiebler D, Gavranović B, and Wilson P. Category Theory in Machine Learning. 2021. arXiv: 2106.07032 [cs.LG]. Available from: https://arxiv.org/abs/2106.07032
3. Plaut DC, Nowlan SJ, and Hinton GE. Experiments on Learning by Back Propagation. 1986. Available from: https://api.semanticscholar.org/CorpusID:15150815
4. Cruttwell GSH, Gavranović B, Ghani N, Wilson P, and Zanasi F. Categorical Foundations of Gradient-Based Learning. 2021. arXiv: 2103.01931 [cs.LG]. Available from: https://arxiv.org/abs/2103.01931
5. Fong B, Spivak DI, and Tuyéras R. Backprop as Functor: A compositional perspective on supervised learning. 2019. arXiv: 1711.10455 [math.CT]. Available from: https://arxiv.org/abs/1711.10455
6. Fong B and Johnson M. Lenses and Learners. 2019. arXiv: 1903.03671 [cs.LG]. Available from: https://arxiv.org/abs/1903.03671
7. Elliott C. The simple essence of automatic differentiation. 2018. arXiv: 1804.00746 [cs.PL]. Available from: https://arxiv.org/abs/1804.00746
8. Pickering M, Gibbons J, and Wu N. Profunctor Optics: Modular Data Accessors. The Art, Science, and Engineering of Programming 2017 Apr; 1. DOI: 10.22152/programming-journal.org/2017/1/7. Available from: http://dx.doi.org/10.22152/programming-journal.org/2017/1/7
9. Kmett E. Control.Lens.Lens — lens and related combinators. Accessed: 2026-01-20. 2025. Available from: https://hackage-content.haskell.org/package/lens-5.3.5/docs/src/Control.Lens.Lens.html#lens
10. Foster JN, Greenwald MB, Moore JT, Pierce BC, and Schmitt A. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View Update Problem. *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM Press, 2005 :233–46. DOI: 10.1145/1040305.1040325

11. Bancilhon F and Spyratos N. Update semantics of relational views. ACM Transactions on Database Systems 1981; 6:557–75

12. Peyton Jones S. Lenses: Compositional Data Access and Manipulation. Talk at Haskell eXchange 2013. Video recording archived by the Internet Archive; accessed 2026-01-20. 2013. Available from: https://archive.org/details/lenses-compositional-data-access-and-manipulation-simon-peyton-jones-at-haskell-

13. Peyton Jones S et al. Haskell 2010 Language Report. Haskell.org. 2010. Available from: https://www.haskell.org/onlinereport/haskell2010/

14. The Glasgow Haskell Compiler Team. Functor — Control.Monad. Accessed: 2026-01-20. 2025. Available from: https://hackage-content.haskell.org/package/base-4.22.0.0/docs/Control-Monad.html#t:Functor

15. The GHC Developers. GHC.Internal.Data.Functor.Identity — Documentation. https://hackage-content.haskell.org/package/ghc-internal-9.1401.0/docs/src/GHC.Internal.Data.Functor.Identity.html#Identity. Accessed: 2026-01-20, version ghc-internal-9.1401.0. 2026

16. The GHC Developers. GHC.Internal.Data.Functor.Const — Documentation. https://hackage-content.haskell.org/package/ghc-internal-9.1401.0/docs/src/GHC.Internal.Data.Functor.Const.html#Const. Accessed: 2026-01-20, version ghc-internal-9.1401.0. 2026

17. Breitner J, Eisenberg RA, Peyton Jones S, and Weirich S. Safe zero-cost coercions for Haskell. *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*. ICFP '14. Association for Computing Machinery, 2014 :189–202. DOI: 10.1145/2628136.2628141

18. Kingma DP and Ba J. Adam: A Method for Stochastic Optimization. 2017. arXiv: 1412.6980 [cs.LG]. Available from: https://arxiv.org/abs/1412.6980

19. He K, Zhang X, Ren S, and Sun J. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. 2015 Feb. arXiv: 1502.01852 [cs.CV]

20. O'Sullivan B. criterion: Robust, reliable performance measurement and analysis. https://hackage.haskell.org/package/criterion. Haskell package, Hackage. 2009

21. Ruiz A et al. hmatrix: Numeric Linear Algebra and Matrix Computations. https://hackage.haskell.org/package/hmatrix. Haskell package, Hackage

22. Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, Desmaison A, Köpf A, Yang E, DeVito Z, Raison M, Tejani A, Chilamkurthy S, Steiner B, Fang L, Bai J, and Chintala S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems 32*. 2019. Available from: `https://arxiv.org/abs/1912.01703`

23. Frison G, Sartor T, Zanelli A, and Diehl M. The BLAS API of BLASFEO: Optimizing Performance for Small Matrices. ACM Transactions on Mathematical Software 2020; 46. DOI: `10.1145/3378539`